

B-Tree Insertion and Deletion

Updates

- Next Quiz (4) is on Thursday (**Thu Apr 16**)
- HW 2 Deadline **Extended** (Note Qs that require today's content)
 - **Tue Apr 14**
- Project 2 will be posted today
 - Deadline: ~~Thu Apr 16~~ **Tue Apr 21**
- Also today: **Reflection**

Quiz 3

Question

True Or False

A single disk block can always hold more index entries for a sparse index than that for a dense index.

0.56

Item Difficulty ⓘ

0.56/1

PTS

Mean Earned Points

1/1 PTS

Median Earned Points

0.53

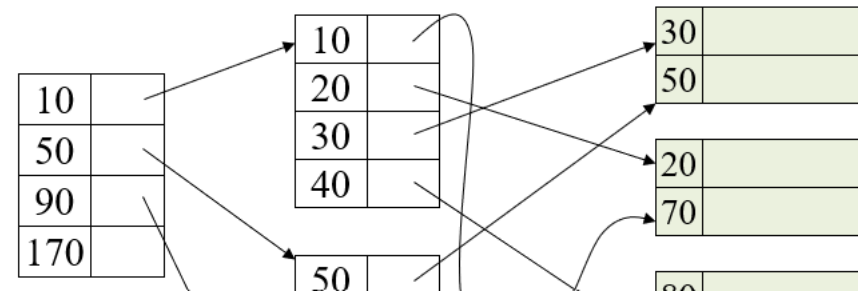
Discrimination Index ⓘ

0.05

Corrected Item-Total Correlation ⓘ

Answer Frequency Summary

	Answer	Respondents	Percentage
✕	True	25	44%
✓	False	32	56%



Quiz 3

Question

True Or False

Considering one-level indexes (i.e., no multi-layer index allowed), a relation with several attributes can have at most one sparse index and possibly several dense indexes simultaneously.

0.61

Item Difficulty ⓘ

0.61/1

PTS

Mean Earned Points

1/1 PTS

Median Earned Points

0.42

Discrimination Index ⓘ

0.03

Corrected Item-Total Correlation Coefficient ⓘ

Answer Frequency Summary

	Answer	Respondents	Percentage
✓	True	35	61%
✗	False	22	39%

name age sal

bob	12	10
cal	11	80
joe	12	20
sue	13	20

Data records sorted by *name*

11
12
12
13

<age>

10
20
20
80

<sal>

Quiz 3

Question

Multiple Choice

In External Sorting, assume the number of memory buffers is 5. What is the maximum size of a relation that can be fully sorted in one preparation phase plus two merge phases?

0.72

Item Difficulty ⓘ

0.72/1

PTS

Mean Earned
Points

1/1 PTS

Median Earned
Points

0.58

Discrimination Index ⓘ ^

0.36

Corrected Item-Total
Correlation Coefficient ⓘ

Answer Frequency Summary

	Answer	Respondents	Percentage
✘	60	6	11%
✔	80	41	72%
✘	120	8	14%
✘	150	2	4%

Quiz 3

Multiple Choice

In External Sorting, assume the number of memory buffers is 5. What is the maximum size of a relation that can be fully sorted in one preparation phase plus two merge phases?

B = number of memory buffers

k = number of merge phases

N = maximum number of blocks in the relation

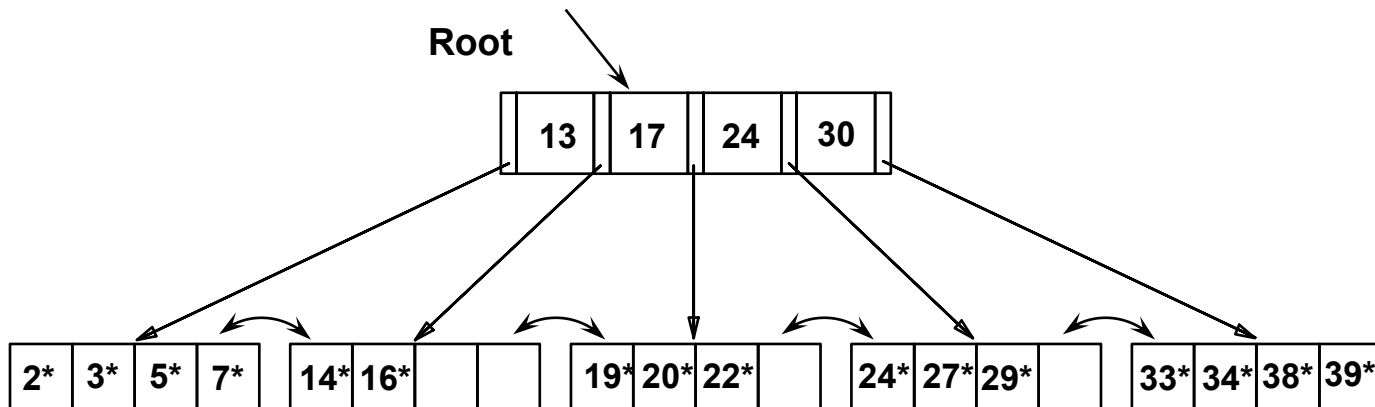
Initial run: N/B ; Merge run reduction: $(B-1)$

$$N \leq 5 \cdot (5 - 1)^2$$

$$N \leq 80$$

B+ Tree

- A B+ Tree is a self-balancing, ordered tree data structure that allows **searches, sequential access, insertions, and deletions in $O(\log n)$** .
→ Generalization of a binary search tree



B-Tree Properties & Invariants

- Each node (1 disk block) holds N keys & N+1 pointers
- Good utilization: Each node is **at least 50% full**
Implementation varies, but we will assume:
 - **Internal nodes** have at least $\lceil (N+1)/2 \rceil$ used **pointers**
 - **Leaves** have at least $\lceil (N+1)/2 \rceil$ used **keys**
 - Exception: **Root** can have as minimum **1 key & 2 pointers**

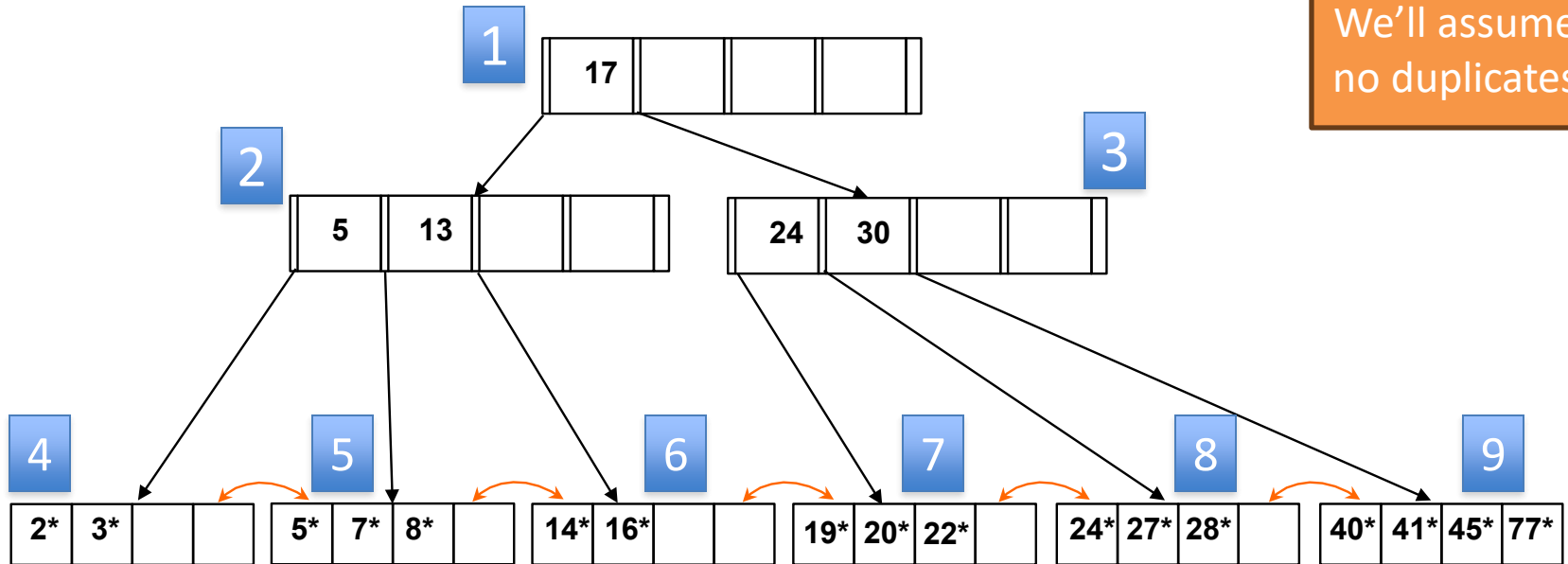
How many records can it index?

- Assume a disk block of 4k bytes = 4096
- Assume indexing integer keys (4 bytes) & each pointer is 8 bytes
- How many entries N can fit in one B-tree node
 - $4N + 8(N+1) = 4096 \quad \rightarrow \quad N = 340$
- **A 3-Level B-Tree (root + 1st and 2nd levels) can index how many records**
(Maximum # of keys)
 - First level (root) $\rightarrow$ 1 Node [340 keys, 341 pointers]
 - 2nd level $\rightarrow$ 341 Nodes [each one contains 340 keys, 341 pointers]
 - 3rd level $\rightarrow 341^2 = 116281$ Nodes {leaf nodes, each has 340 keys}
 - So max number of indexed records is = **116281 x 340 = 39,535, 540 records**

Very efficient & Compact
Structure

B-Tree: Equality Search

We'll assume
no duplicates

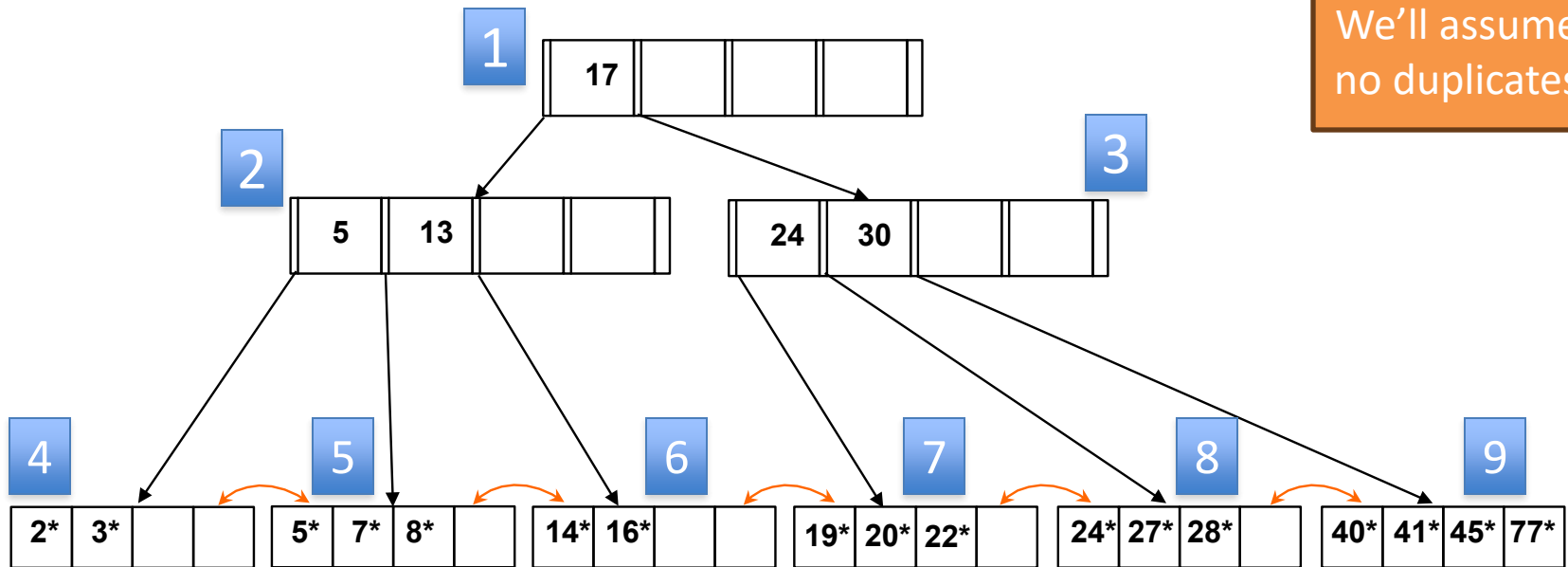


Find key = 1, 3, 41, 89 ?

Find key in [1, 12], [0, 26], [0, 200]?

B-Tree: Equality Search

We'll assume
no duplicates



Find key = 1, 3, 41, 89 ?

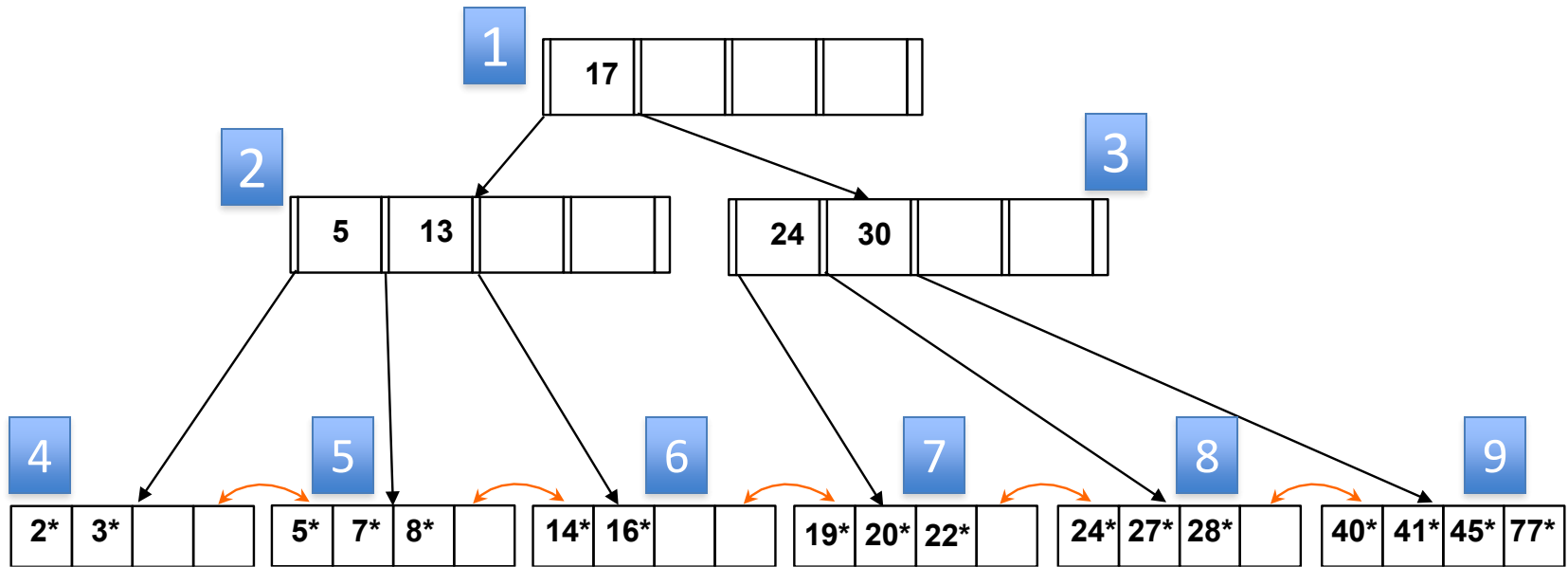
1 → N1, N2, N4

3 → N1, N2, N4, then read the data block

41 → N1, N3, N9, then read the data block

89 → N1, N3, N9

B-Tree: Range Search



Find key in [1, 12], [0, 26], [0, 200]?

[1, 15] → N1, N2, N4, N5, N6

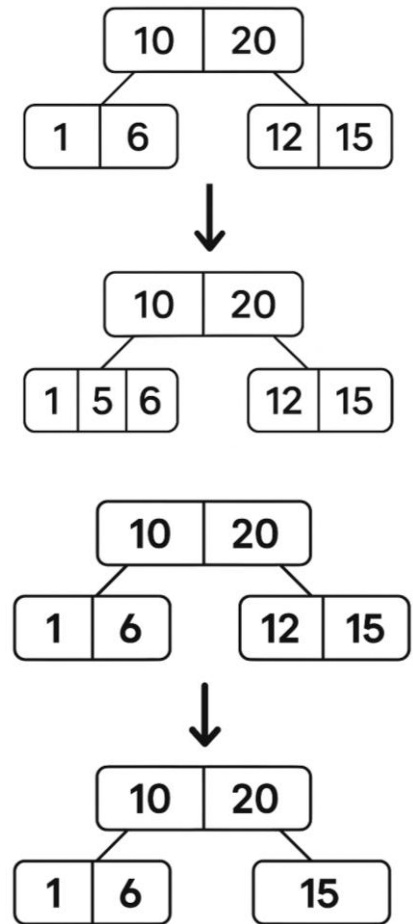
[0, 26] → N1, N2, N4, N5, N6, N7, N8

[0, 200] → N1, N2, N4, ..., N9

That last one... Can we do better?

Learning objectives

- Describe changes in B-tree upon insertion
 - Explain how and why split for leaf and non-leaf is different
- Describe changes in B-tree upon deletion
 - Explain when redistribution of keys is possible vs. when merging nodes is needed



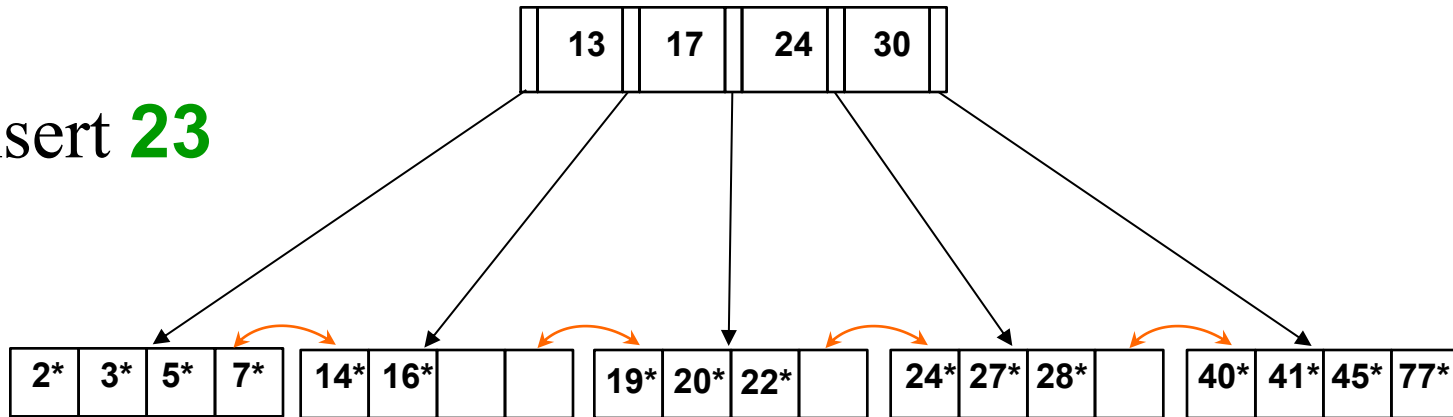
Inserting a Data Entry into a B-Tree

- Find correct leaf **L**. (*Searching*)
- Put data entry onto **L**.
 - If **L** has enough space, *done!*
 - Else, must *split leaf node* **L** (into **L** and a new node L_{NEW})
 - *Redistribute* entries evenly
 - **Copy up** middle key (1st value in L_{NEW}) to parent.
 - *Insert* index entry pointing to L_{NEW} into parent of **L**.
- This can happen recursively
 - To *split internal node*, redistribute entries evenly, but **push up** middle key. (Contrast with leaf splits.)
- Splits “grow” tree; root split increases height.
 - Tree growth: gets *wider* or *one level taller at top*.

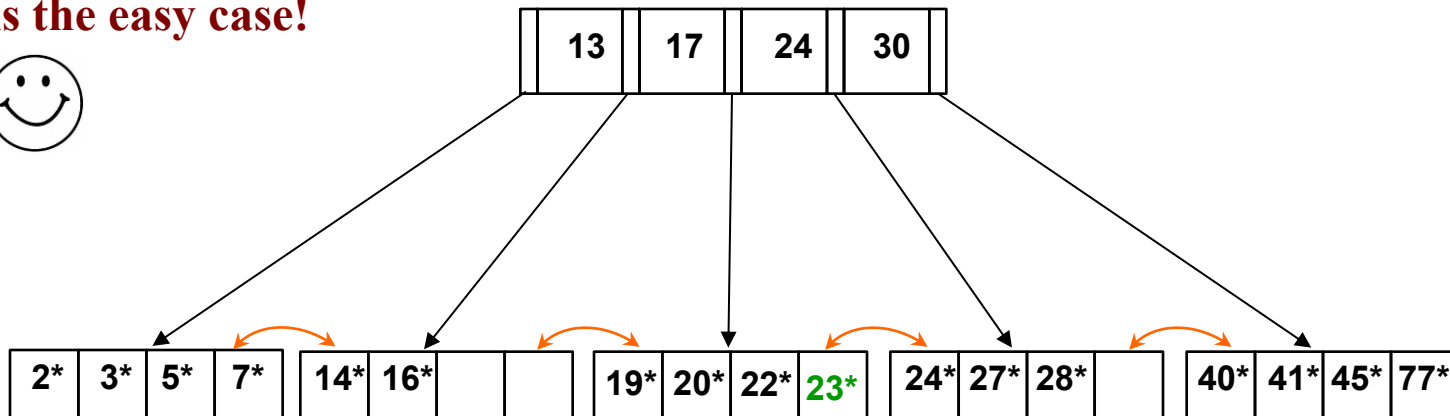
Copy up
vs.
Push up

Updates on B-Trees: Insertion

Insert **23**



This is the easy case!



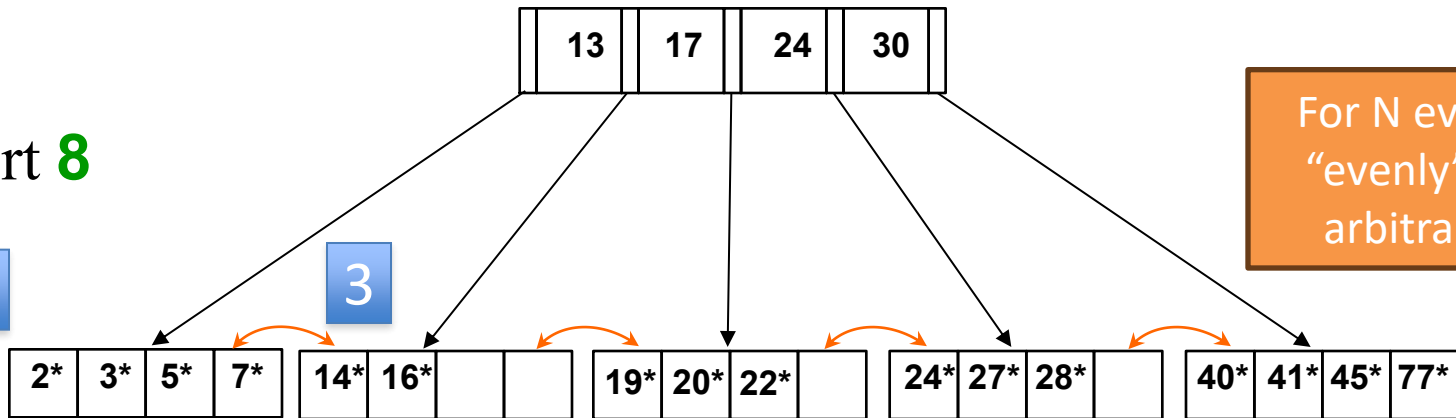
Updates on B⁺-Trees: Insertion

Insert **8**

2

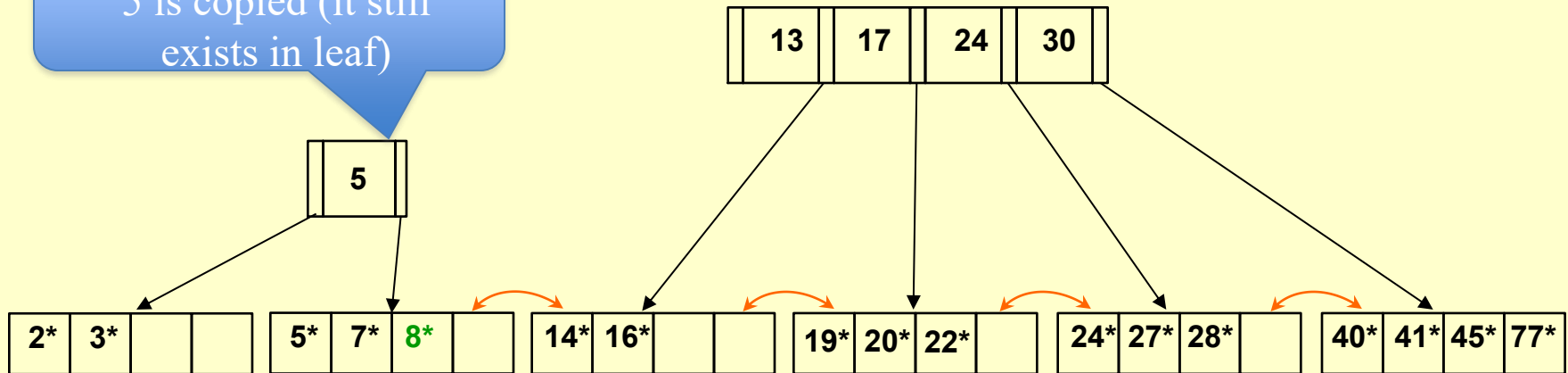
3

For N even,
“evenly” is
arbitrary



Notice:

5 is copied (it still
exists in leaf)



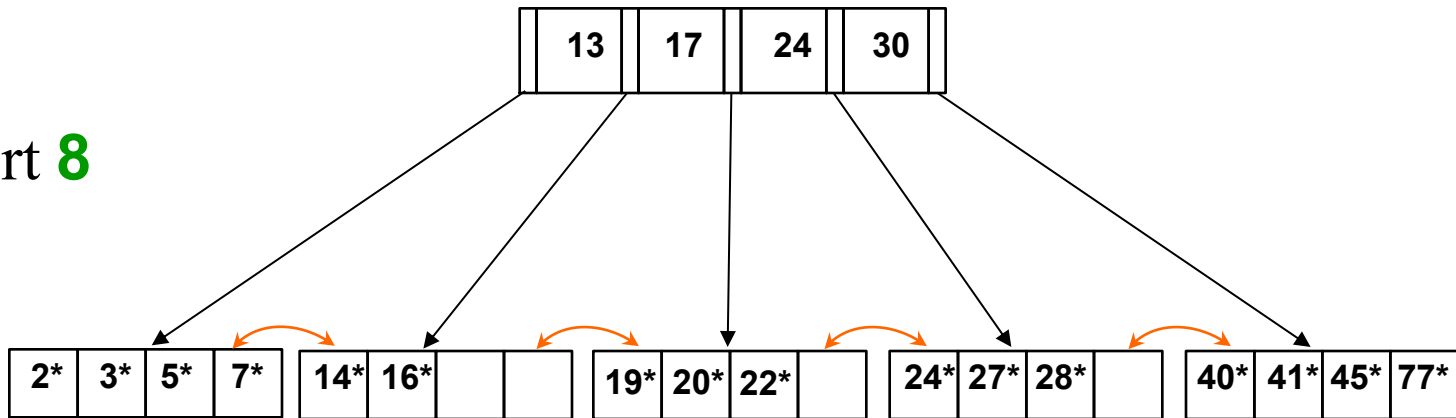
Because insertion causes overflow, we split leaf node into 2 nodes (distribute entries evenly).

“5” is special, since it discriminates between 2 new siblings, so it is **copied up**.

How to insert 5 into parent node?

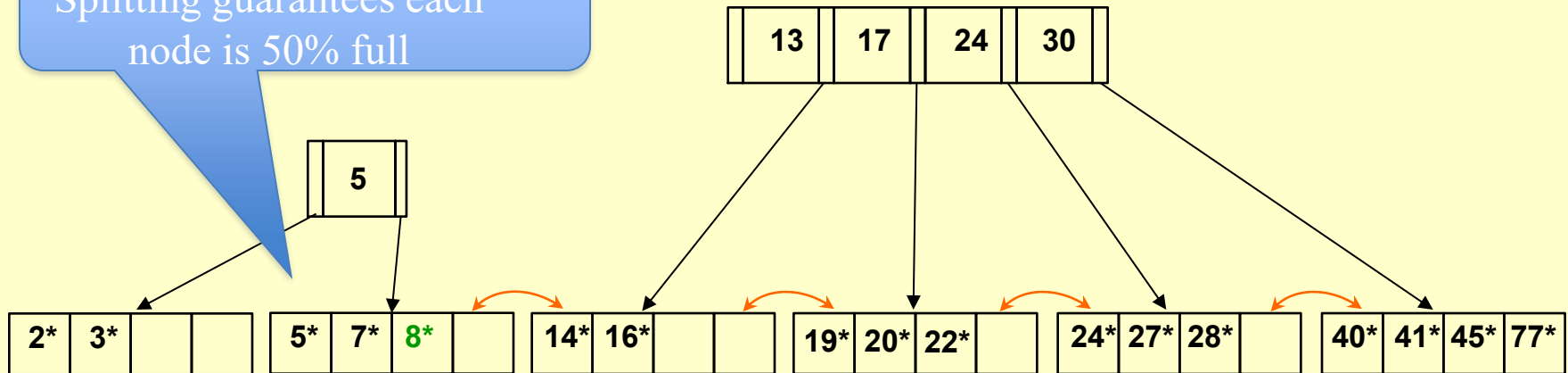
Updates on B⁺-Trees: Insertion

Insert **8**



Notice:

Splitting guarantees each node is 50% full



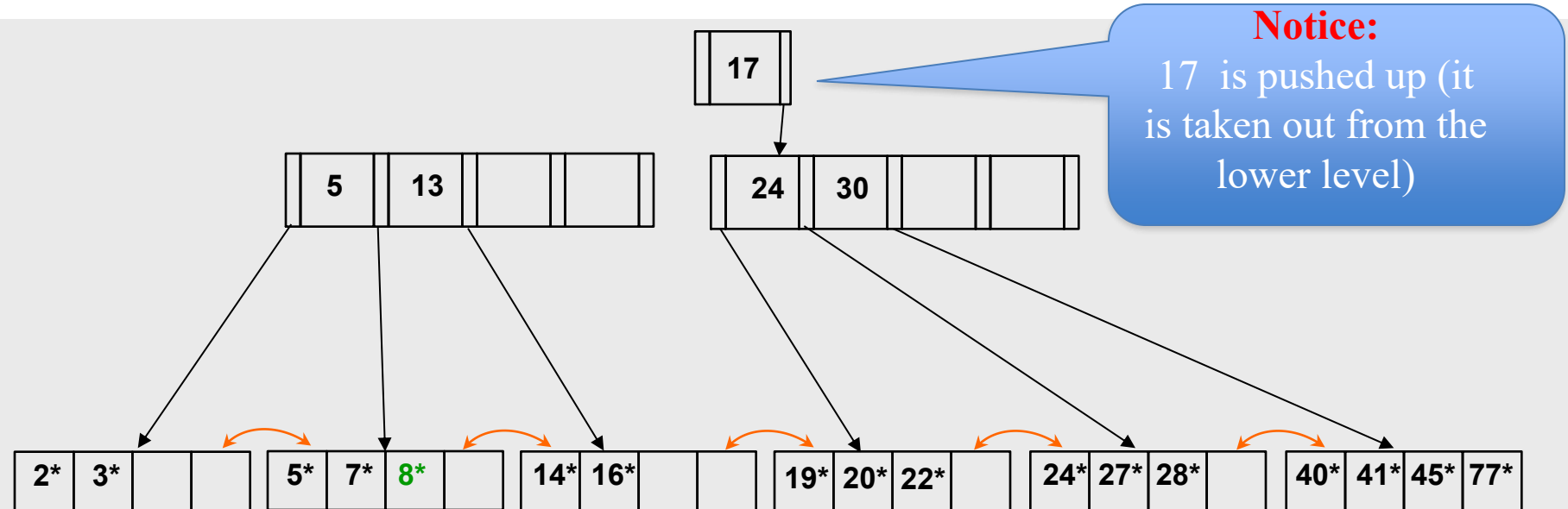
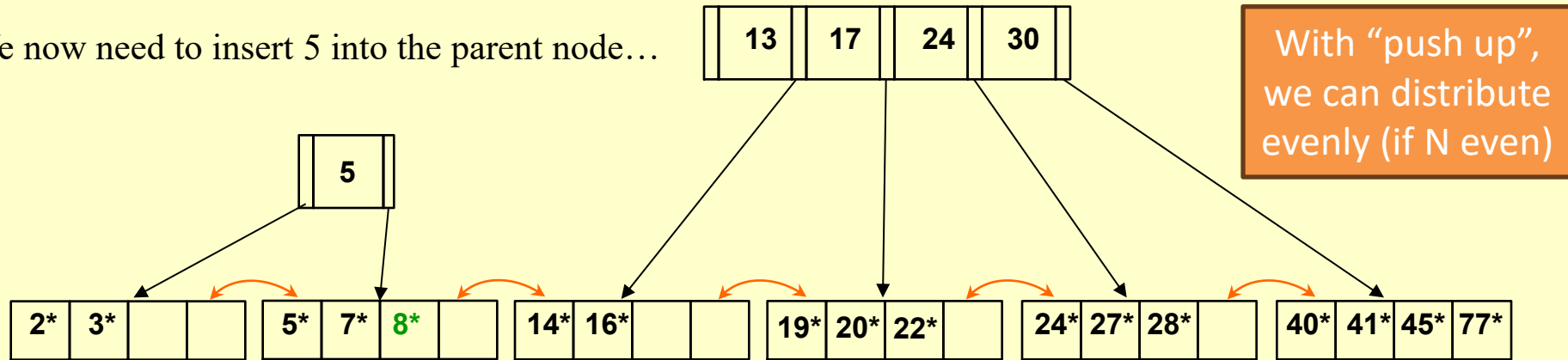
Because insertion causes overflow, we split leaf node into 2 nodes (distribute entries evenly).

“5” is special, since it discriminates between 2 new siblings, so it is **copied up**.

How to insert 5 into parent node?

Updates on B⁺-Trees: Insertion

We now need to insert 5 into the parent node...

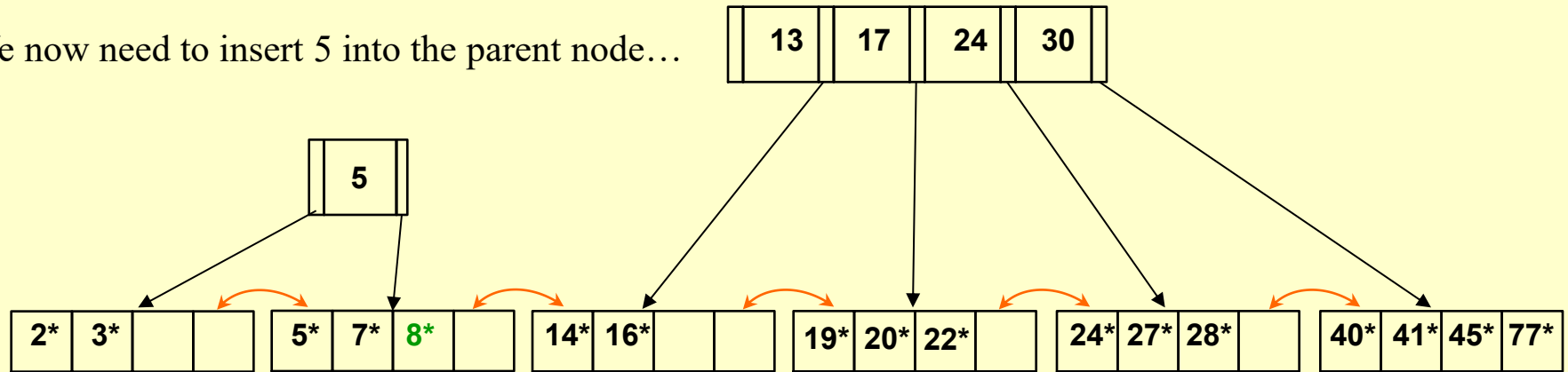


Because insertion causes overflow, we split node into 2 nodes (distribute entries evenly).

"17" is special, since it discriminates between 2 new siblings, so it is **pushed up**.

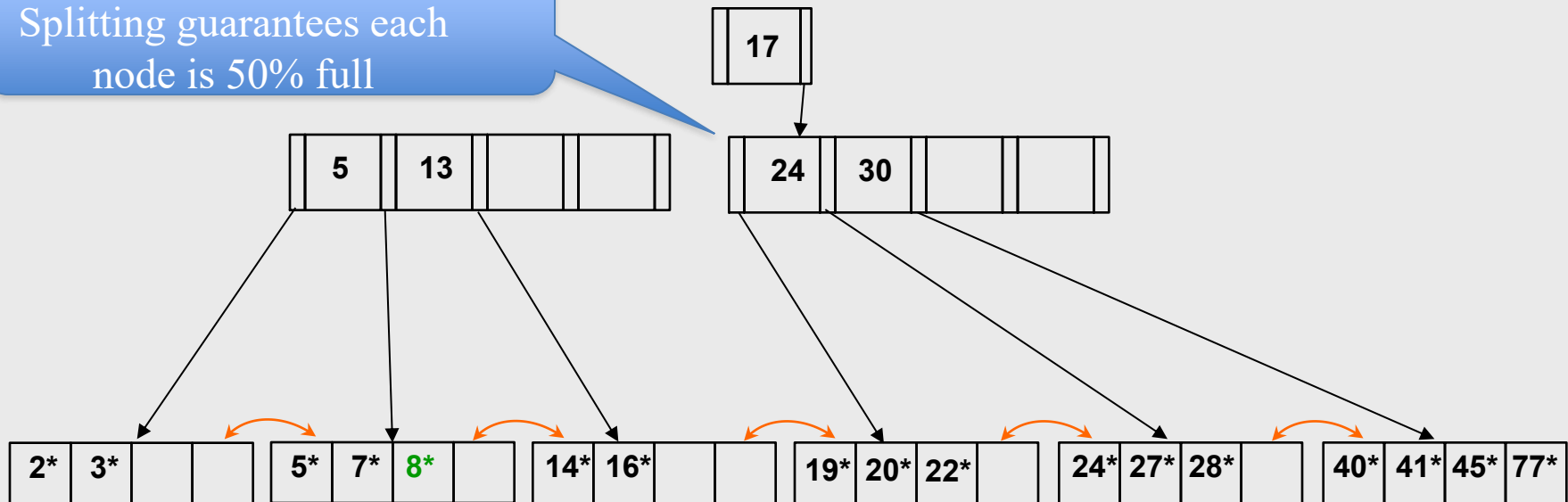
Updates on B⁺-Trees: Insertion

We now need to insert 5 into the parent node...



Notice:

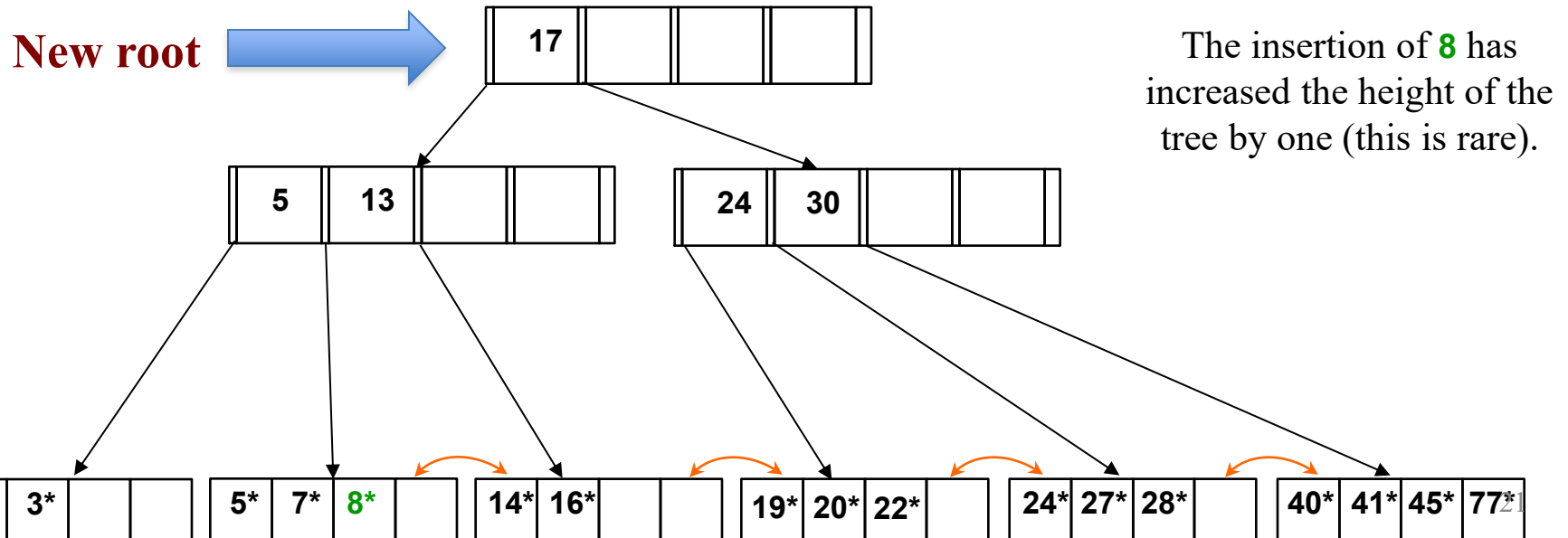
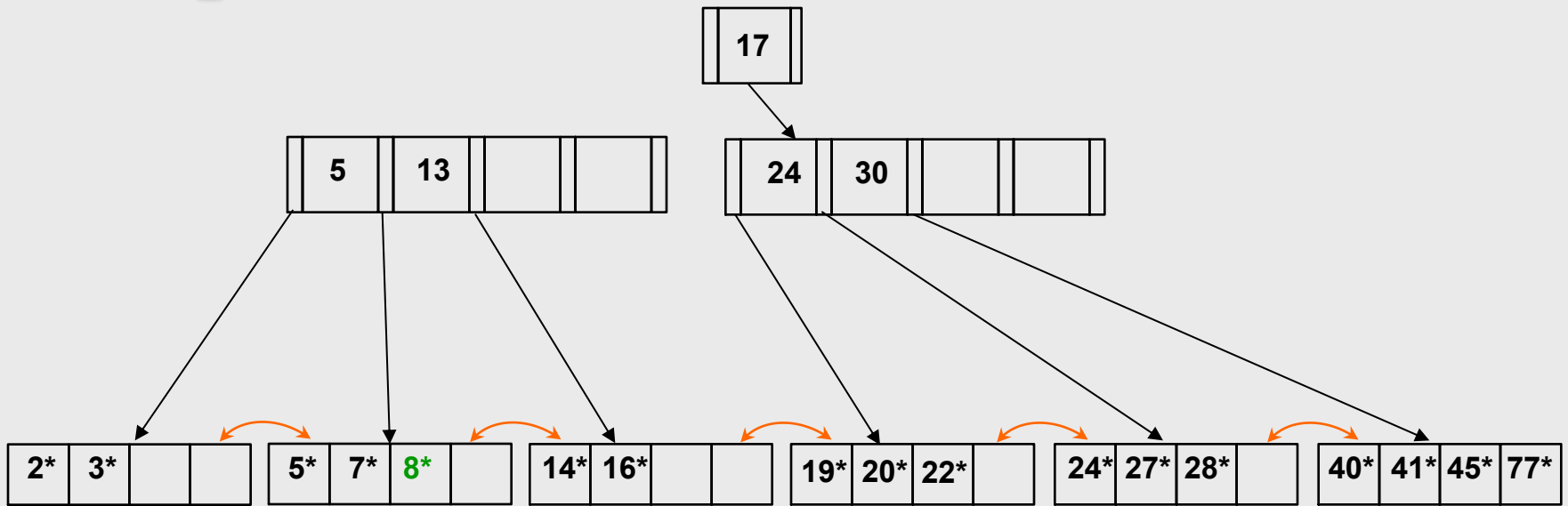
Splitting guarantees each node is 50% full



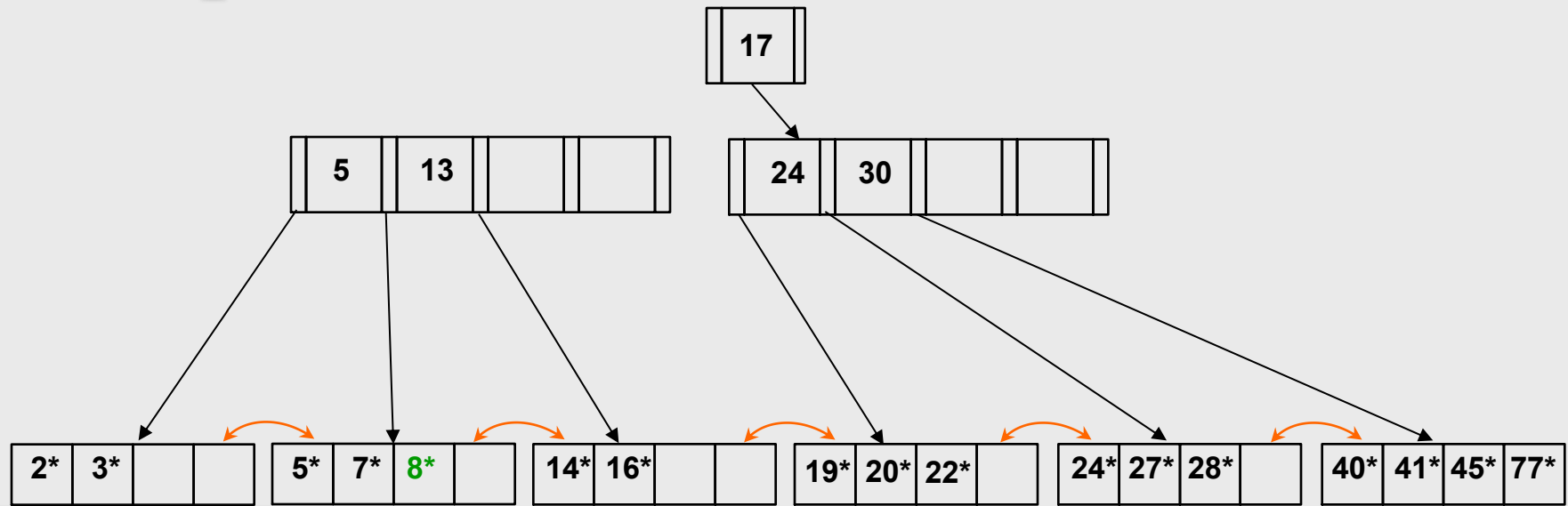
Because insertion causes overflow, we split node into 2 nodes (distribute entries evenly).

“17” is special, since it discriminates between 2 new siblings, so it is **pushed up**.

Updates on B⁺-Trees: Insertion

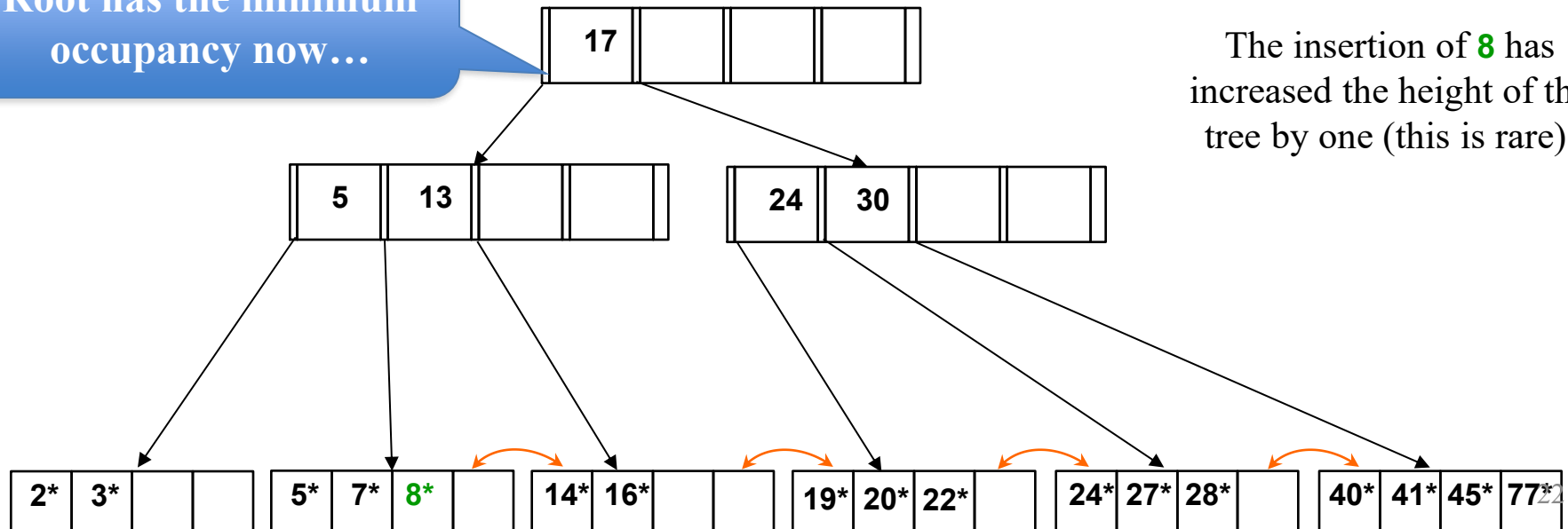


Updates on B⁺-Trees: Insertion



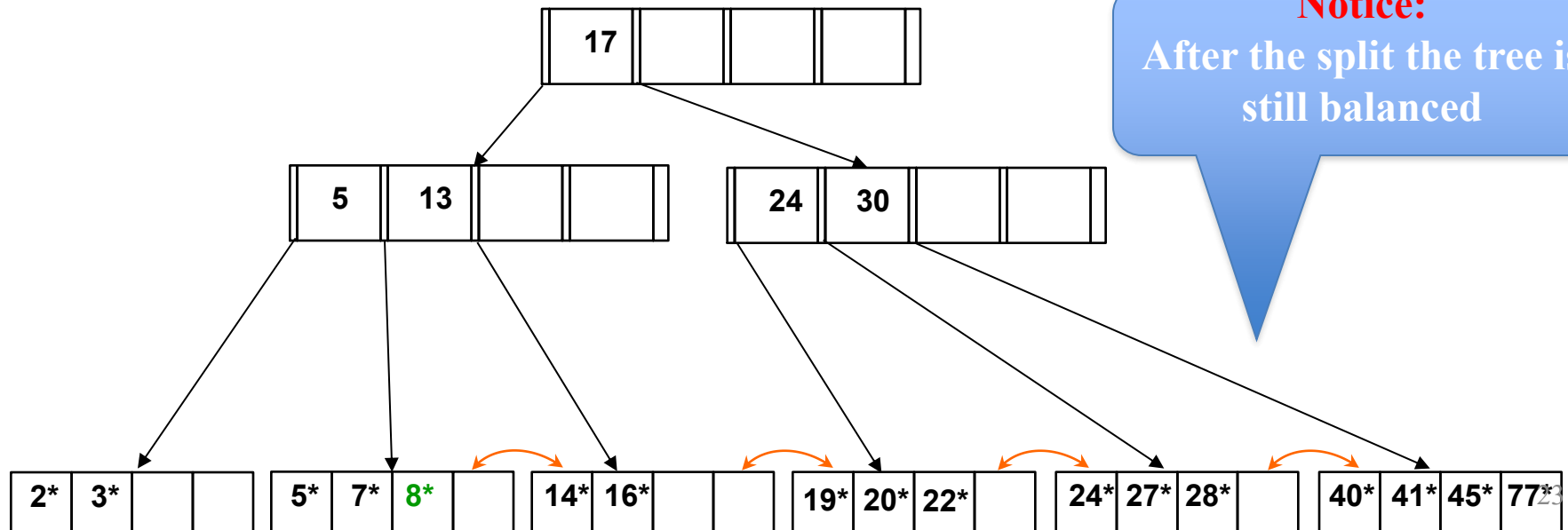
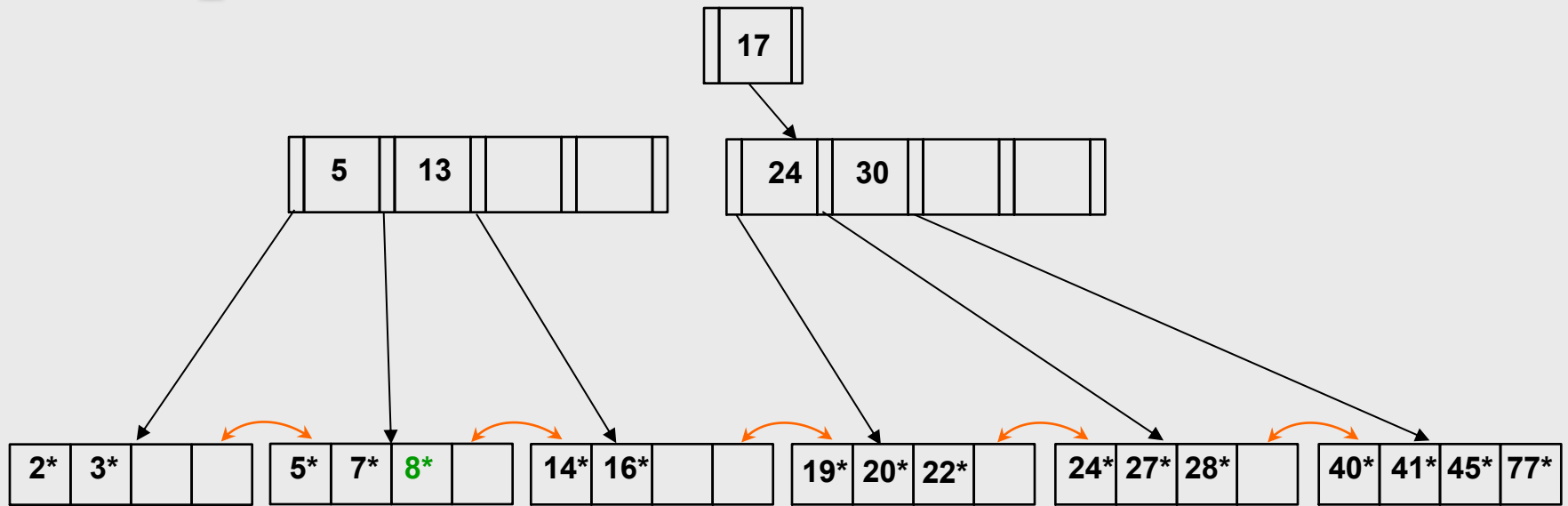
Notice:

Root has the minimum occupancy now...



The insertion of 8 has increased the height of the tree by one (this is rare).

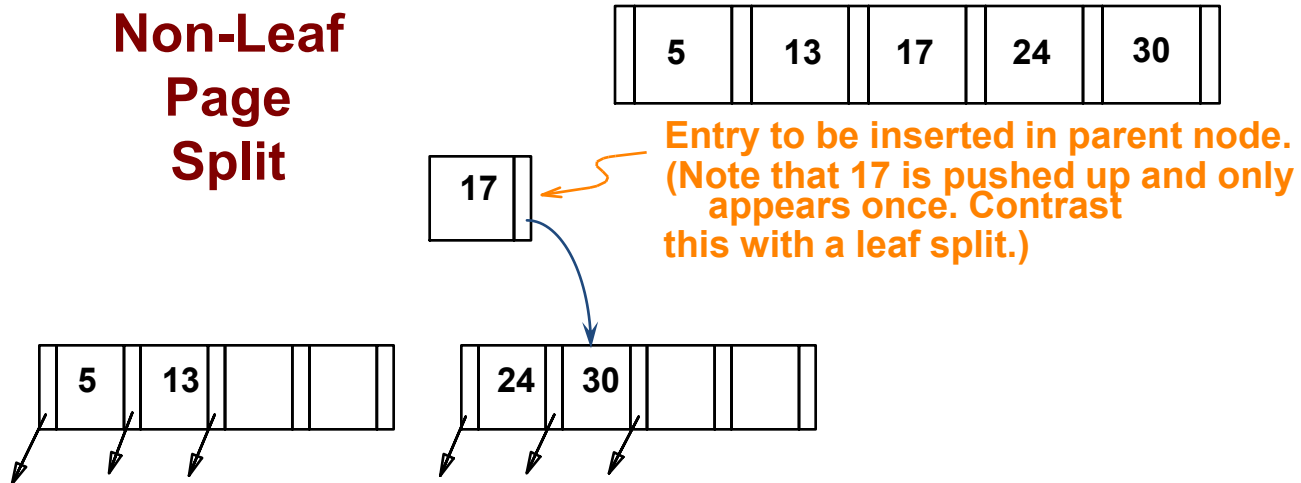
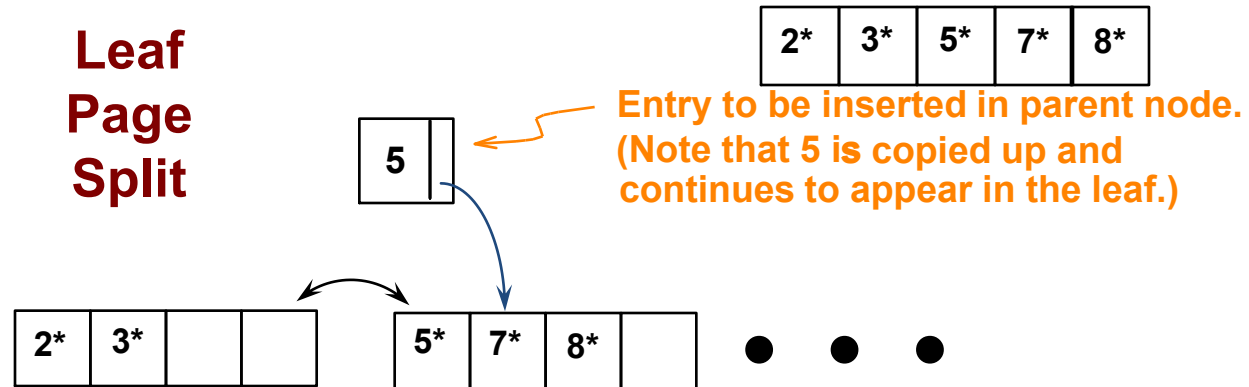
Updates on B⁺-Trees: Insertion



Notice:
After the split the tree is still balanced

Leaf vs. Non-Leaf Page Split

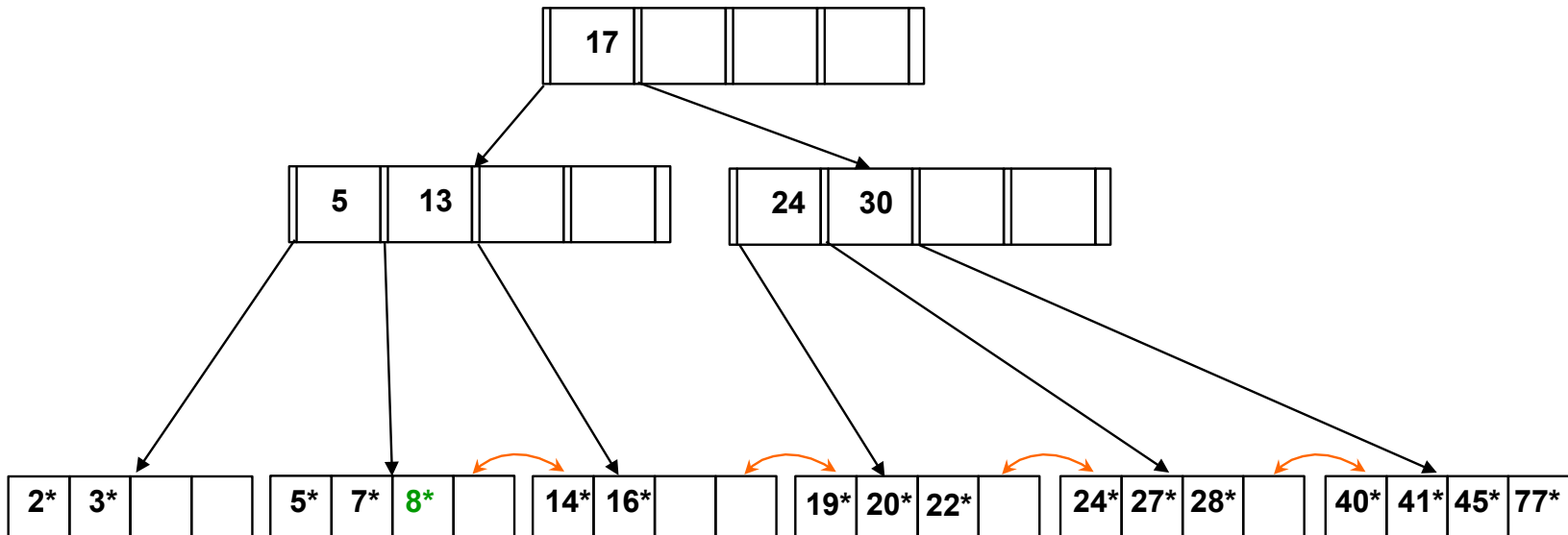
(from previous example of inserting “8”)



Back to Insertion Algorithm

- Find correct leaf **L**. (*Searching*)
- Put data entry onto **L**.
 - If **L** has enough space, *done!*
 - Else, must split **L** (into **L** and a new node **L2**)
 - Redistribute entries evenly, copy up middle key.
 - Insert index entry pointing to **L2** into parent of **L**.
- This can happen recursively
 - To split index node, redistribute entries evenly, but push up middle key. (Contrast with leaf splits.)
- Splits “grow” tree; root split increases height.
 - Tree growth: gets wider or one level taller at top.

Exercise



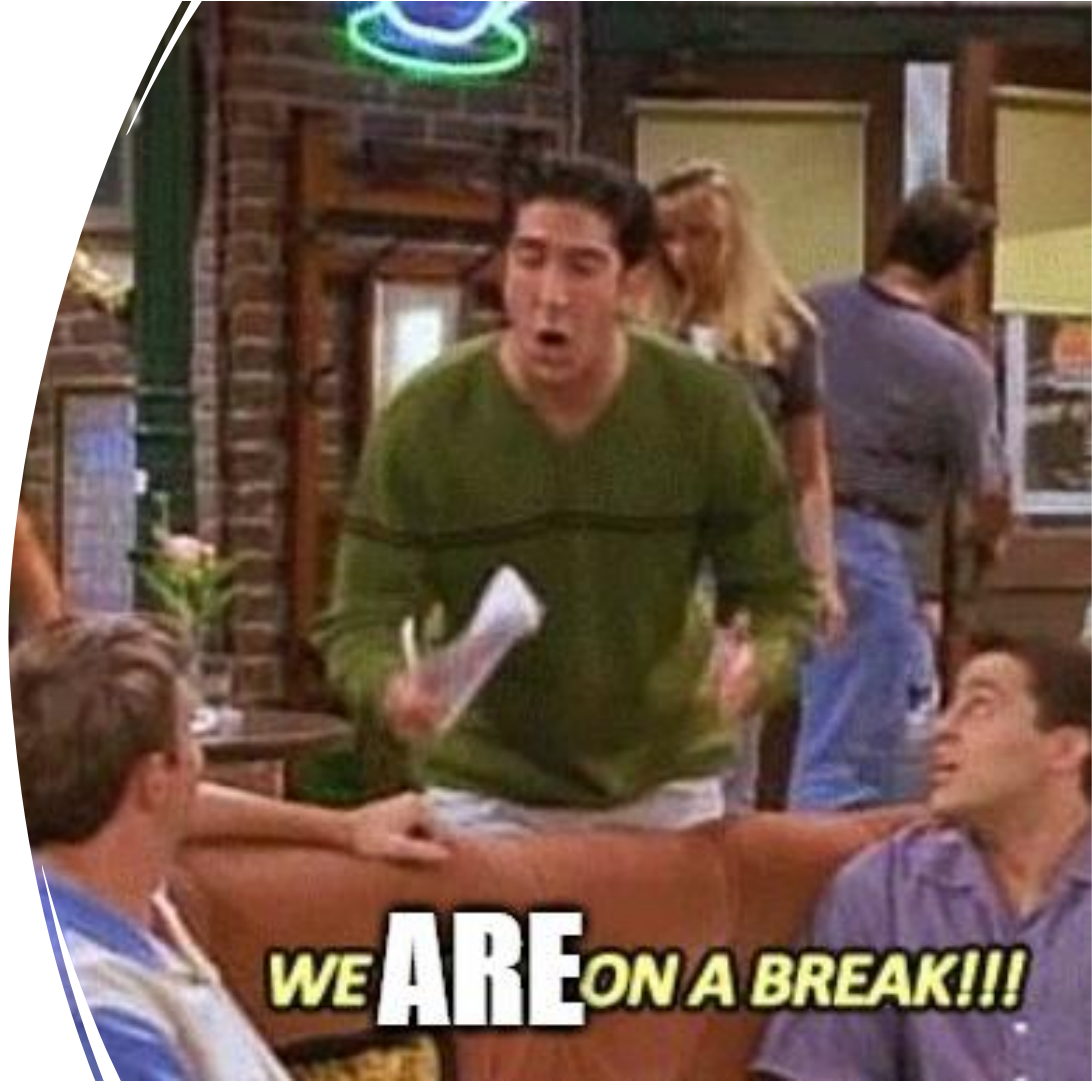
- Insert key 1 ? ➔ [1, 2, 3,]
- Insert key 50 ?
 - [40, 41, 45, 50, 77] ➔ [40, 41, ,] [45, 50, 77,]
 - Copy 45 to parent
 - [24, 30, 45,]

Summary

- Practice the B-Tree insertions
- The split mechanism
 - Ensures each non-root node to be at least 50% full
 - Ensures the tree remains balanced
 - Grows the width of the tree
 - Root splitting increases the tree height

Break

- Now we will take a few minutes break
- **Please login to canvas to complete a reflection**

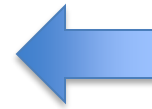


CS4432: Database Systems II

B-Tree Deletion

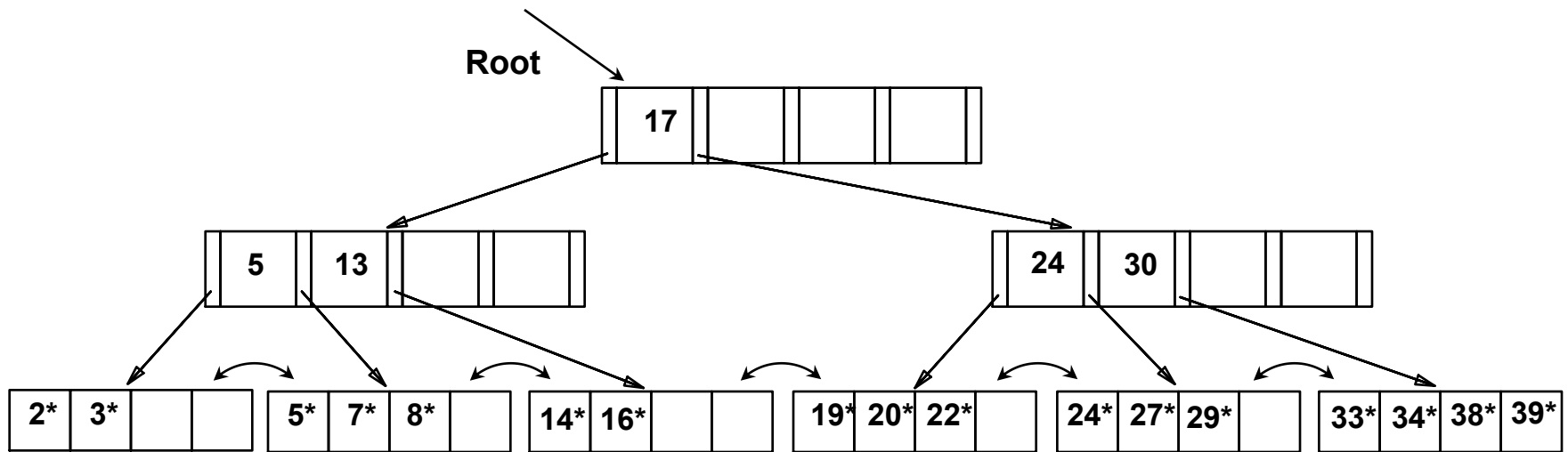
Deleting a Data Entry from a B+ Tree

- Start at root, find **leaf** L where entry belongs. (*Searching*)
- Remove the entry.
 - If L is at least half-full, *done!* 😊
 - If L has less than **the minimum** entries,
 - Try to re-distribute, borrowing from sibling (*adjacent node with same parent as L*).
 - If re-distribution fails, merge L and sibling.
- If merge occurred, must delete entry (pointing to L or sibling) from parent of L .
- Merge could propagate to root, decreasing height.



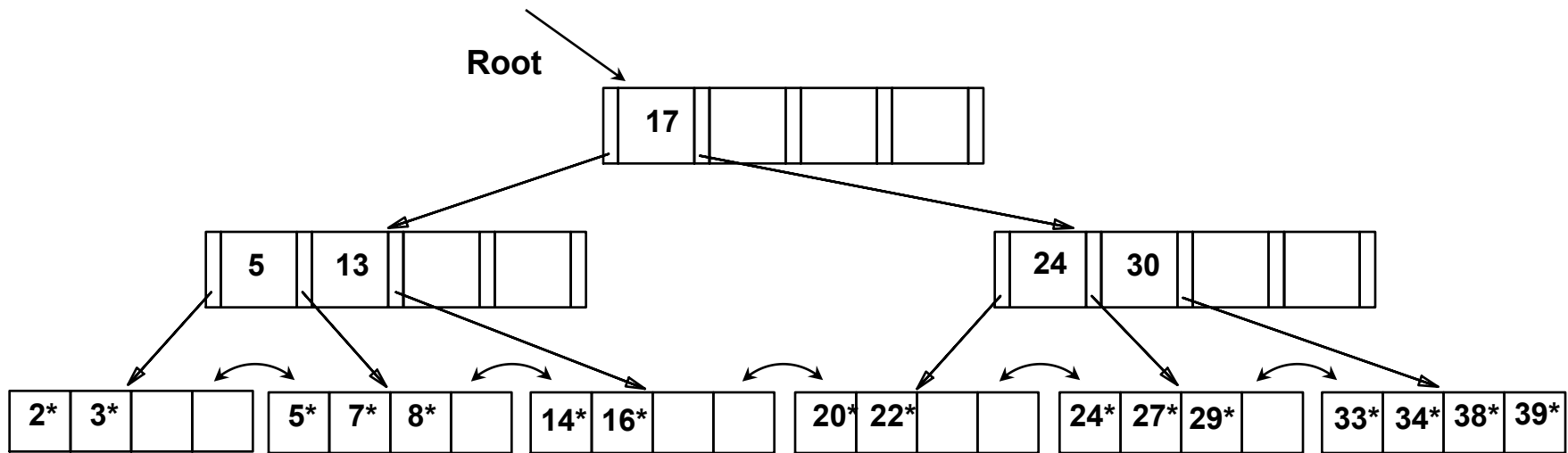
Do not leave empty spaces in the middle

Example Tree: Delete 19*



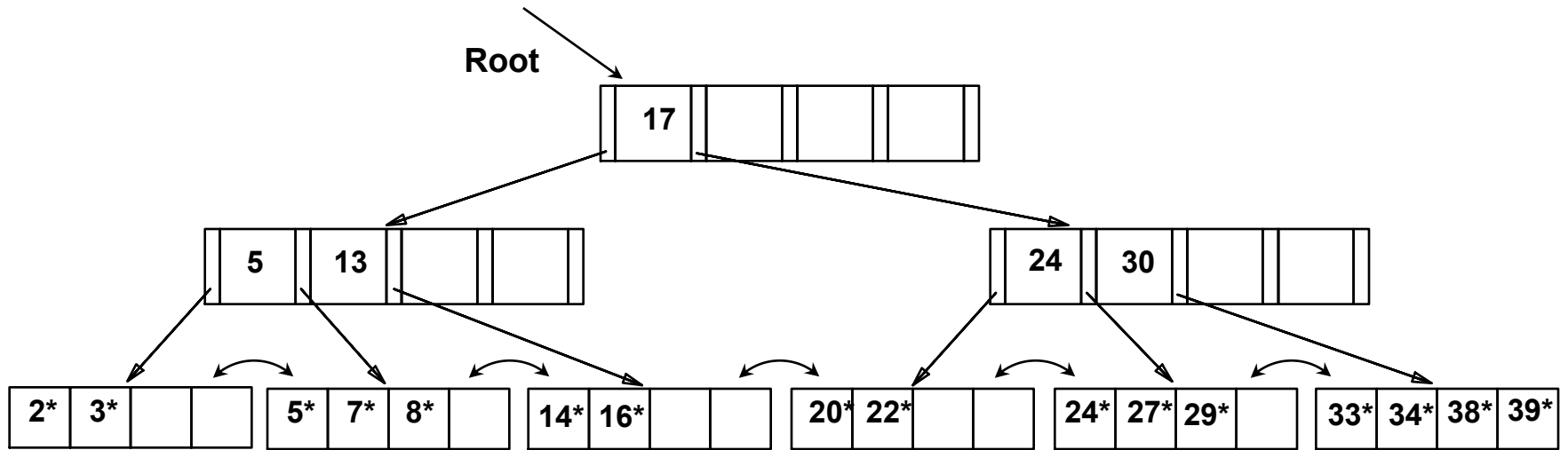
- **Delete 19* is easy. Why?** 😊
 - The leaf node has more than the minimum

Example Tree: After 19* was deleted

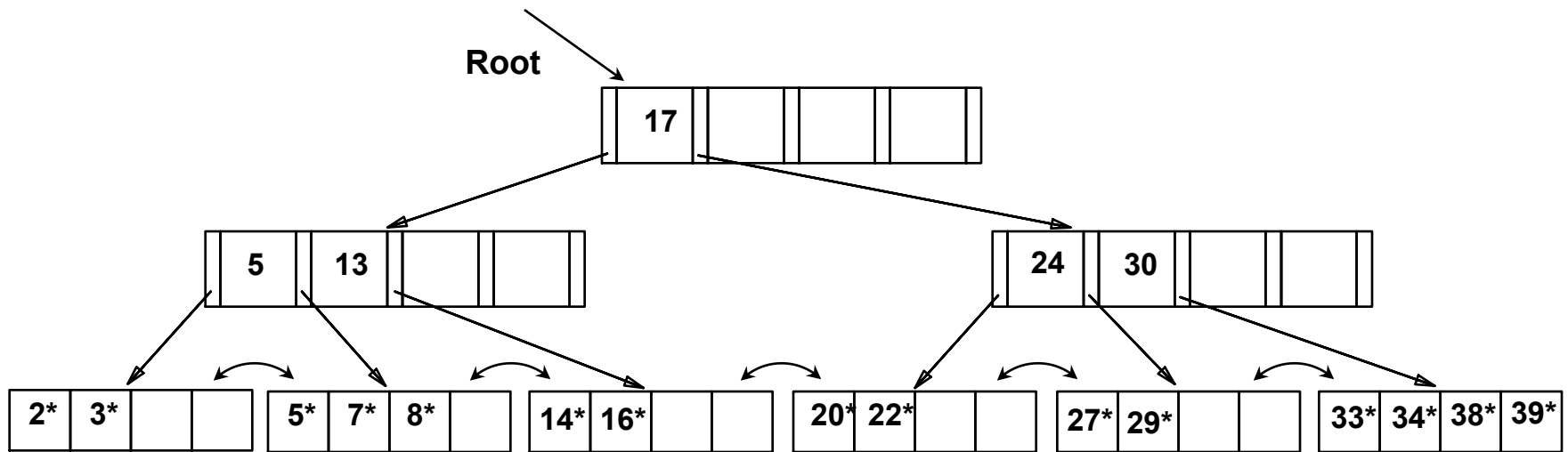


- What else did you observe?
 - Very important in practice...
- **The other keys have shifted...**
 - Remember: empty spaces should be at the end only
Leaf nodes are also index pages!

What if we deleted 24?

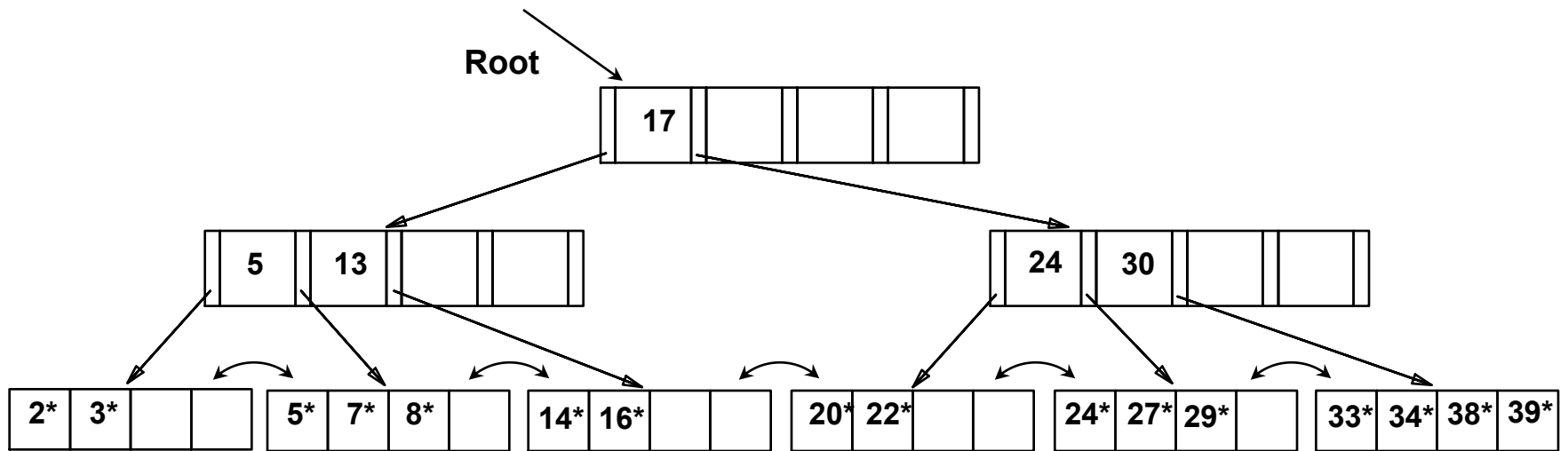


What if we deleted 24?

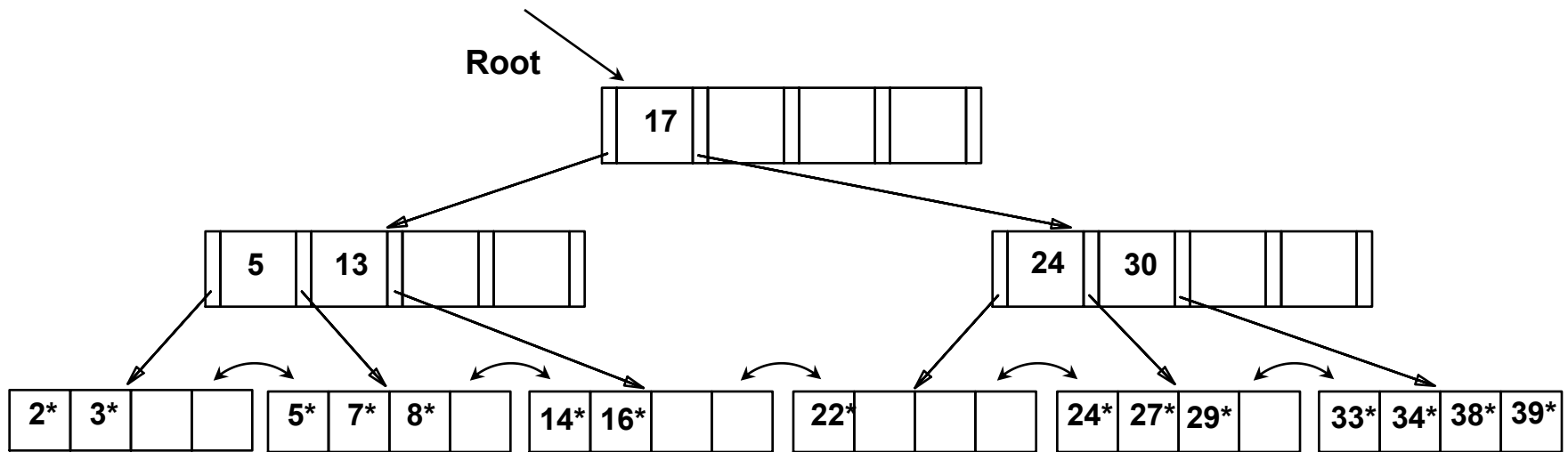


- **The other keys would be shifted...**
 - Remember: empty spaces should be at the end only
- The parent would not be updated!
 - 24 is still a good separator for 1st and 2nd leaves

Example Tree: Delete 20*

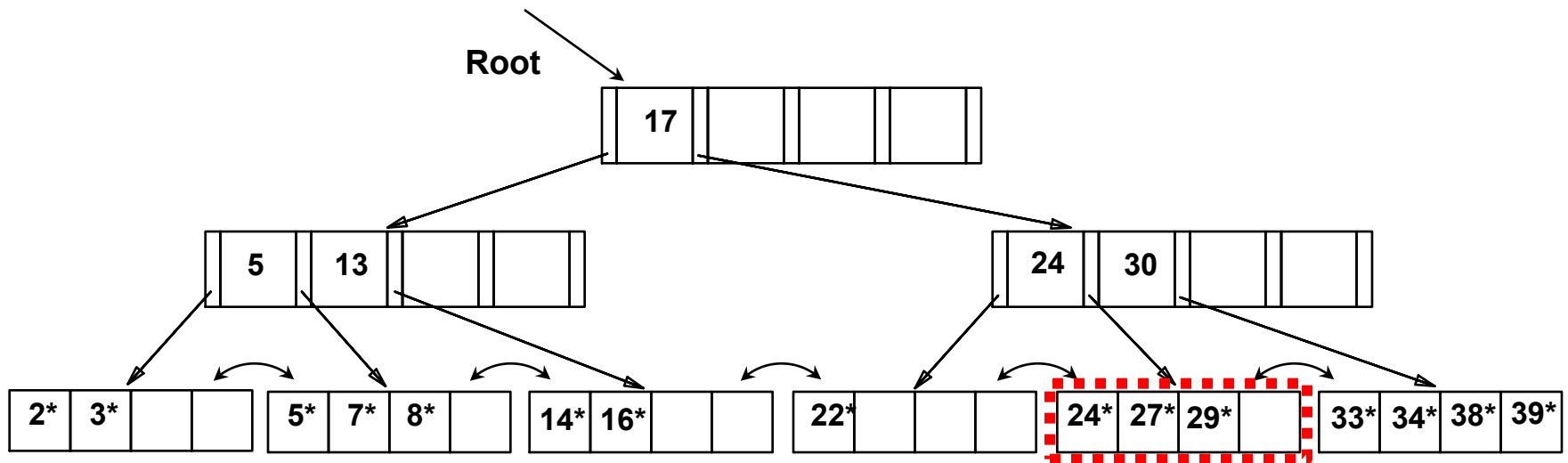


Example Tree: Delete 20*



- After the deletion the 4th node is under the minimum occupancy !!!
- Need to **re-distribute!** How?

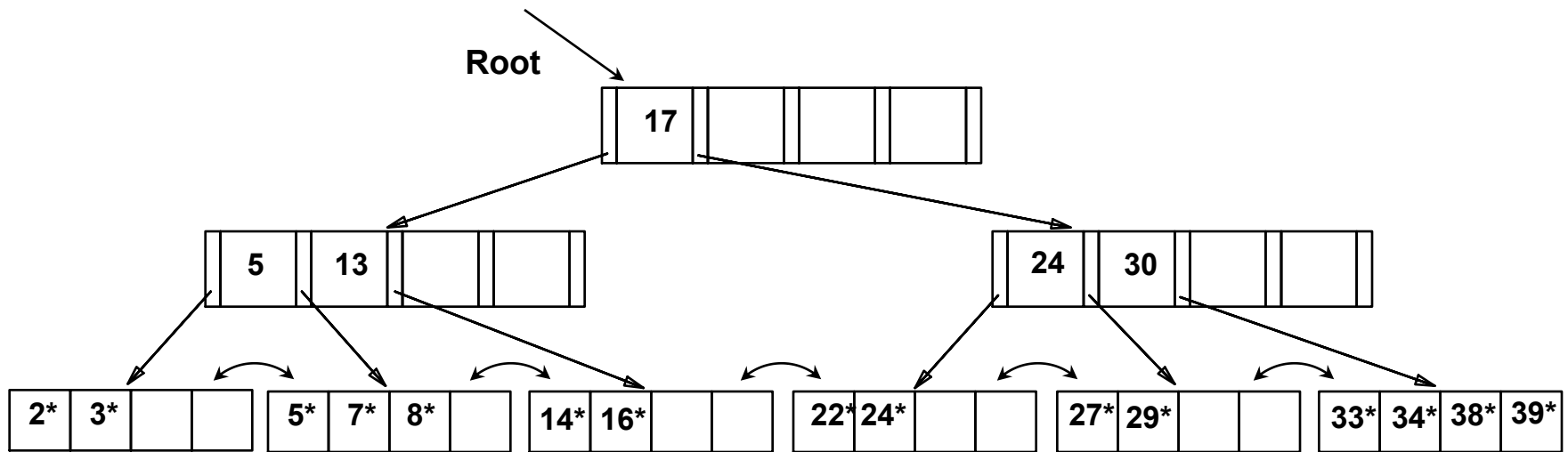
Example Tree: Delete 20*



- Need to **re-distribute!** How?
 - Check either of the two siblings (*only if under the same parent*)
 - Can you borrow from either of them without violating the min occupancy requirements?



Example Tree: Delete 20*

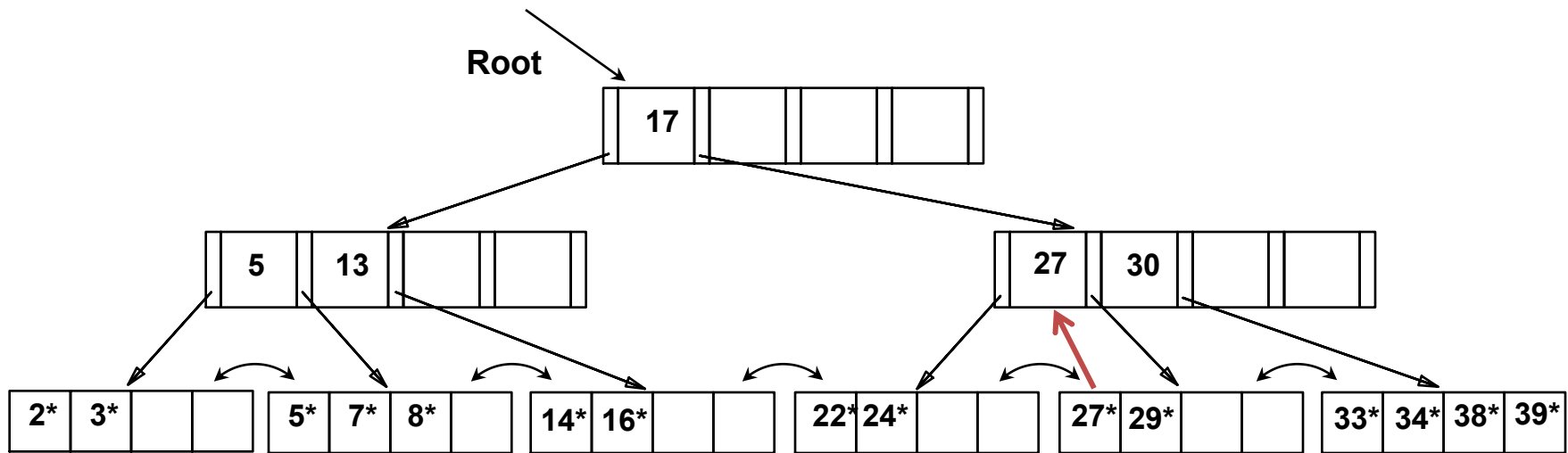


- Need to **re-distribute!** How?
 - Copy 24* into the sibling node (page).



Are we done??

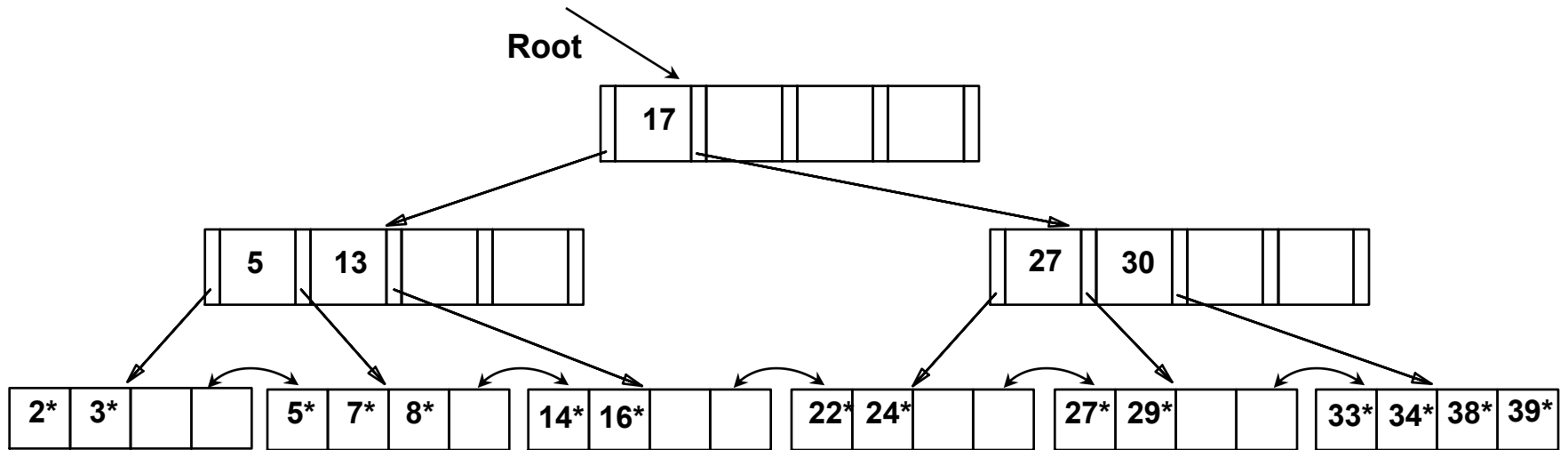
Example Tree: Delete 20*



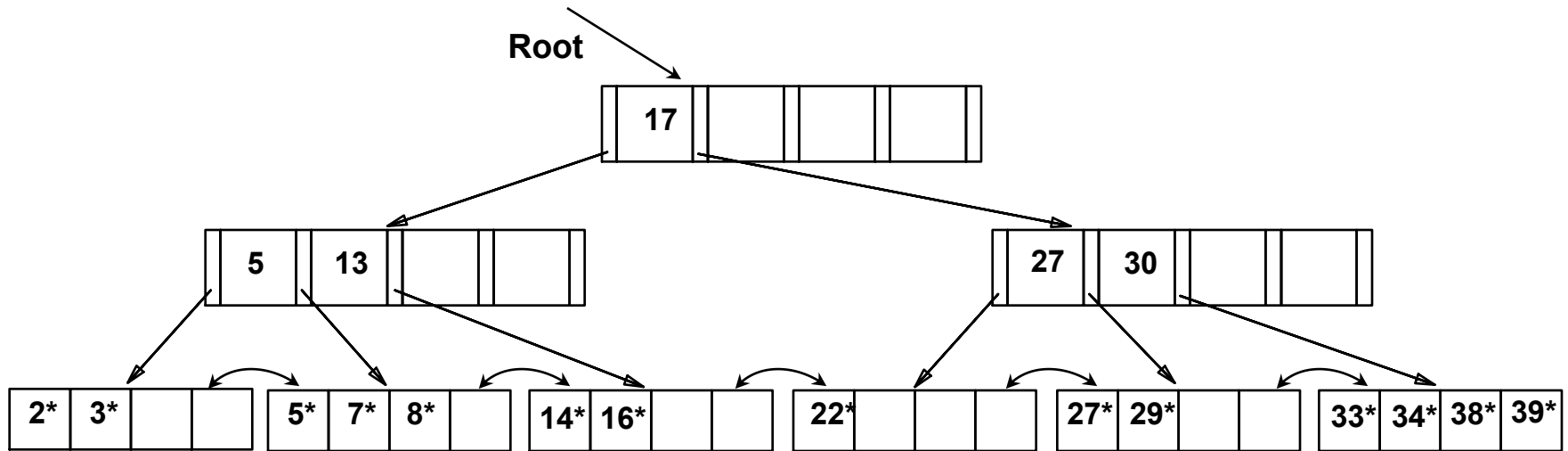
- Need to **re-distribute!** How?
 - Copy 24* into the sibling node (page).
 - Copy *key* 27 into the parent node (page).
 - The parent is only updated if needed!

Notice how
middle key is
copied up

Deleting 24* ...

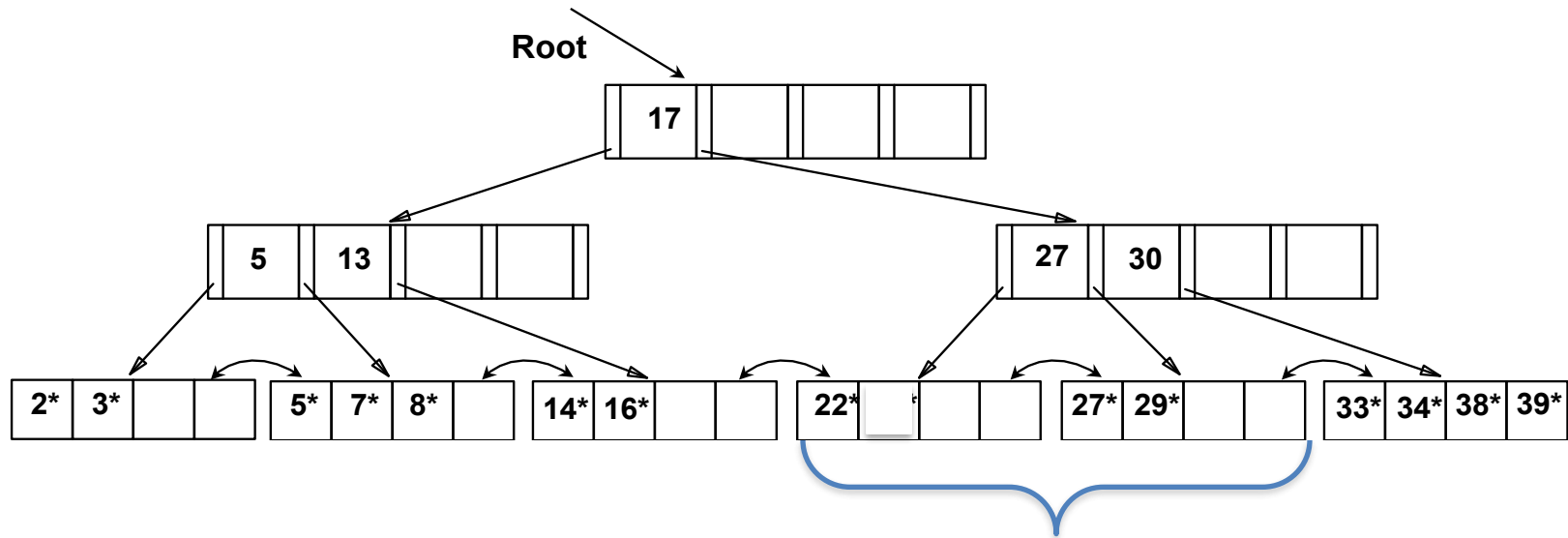


Deleting 24* ...

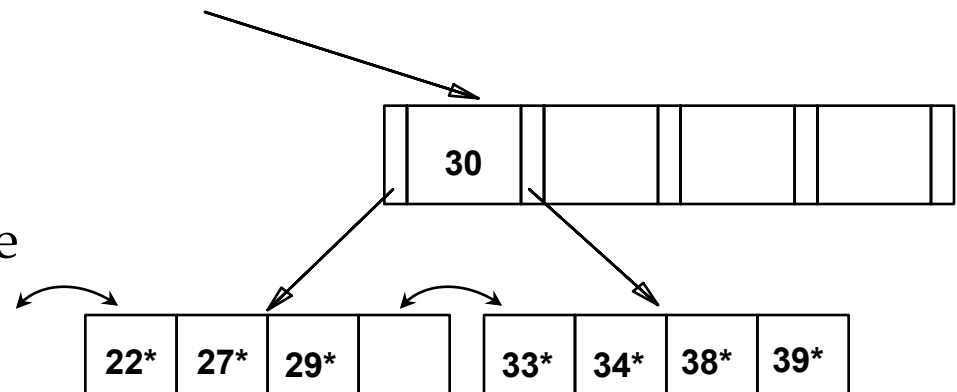


- Borrowing from a sibling will not work !!!

Deleting 24* ...

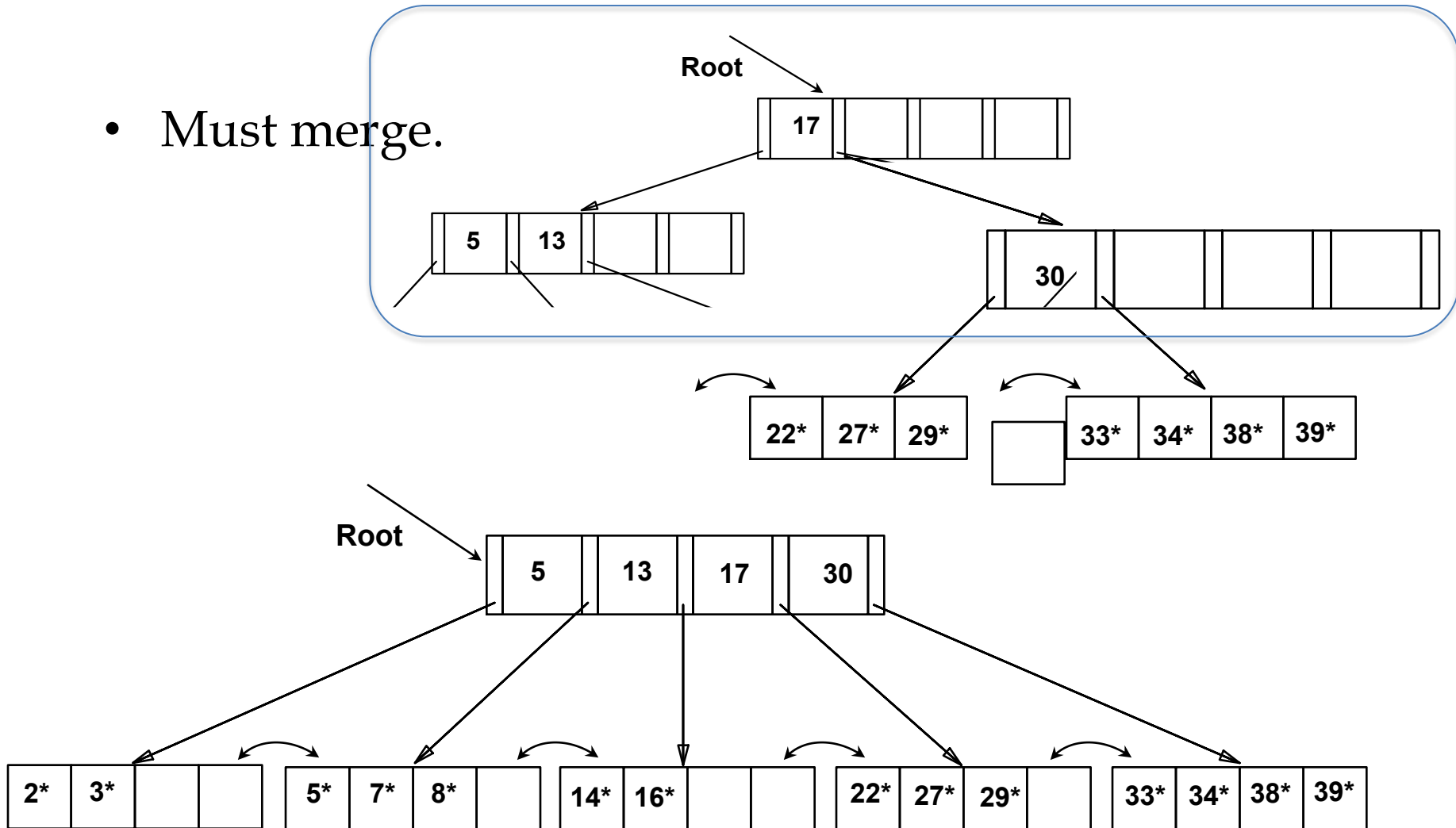


- Must merge (Must be under the same parent)
- Key in parent separating these two nodes must be deleted

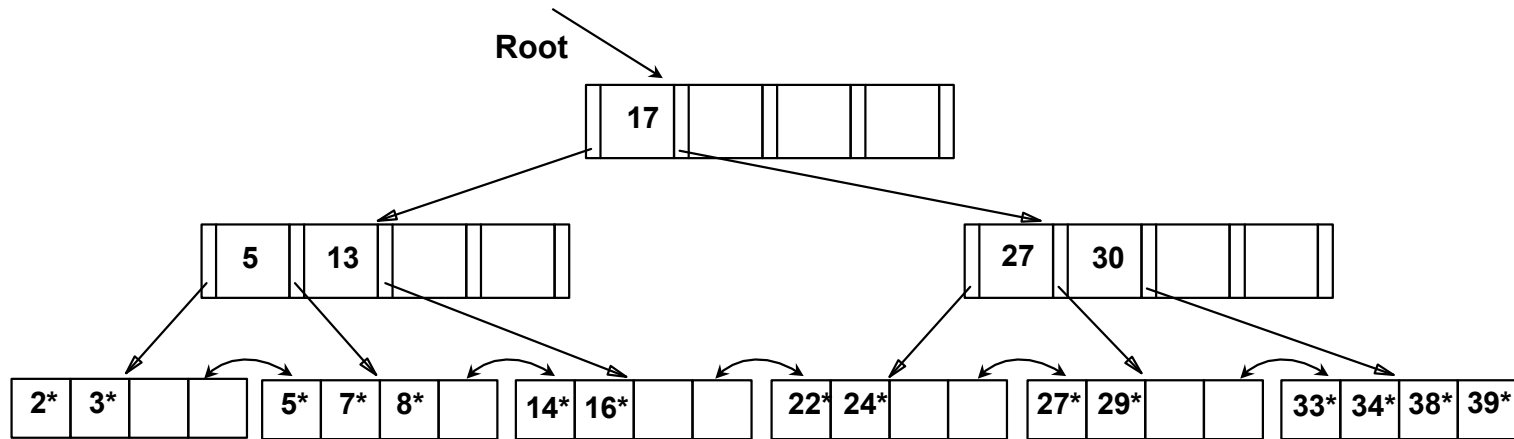


Deleting 24* ...

- Must merge.

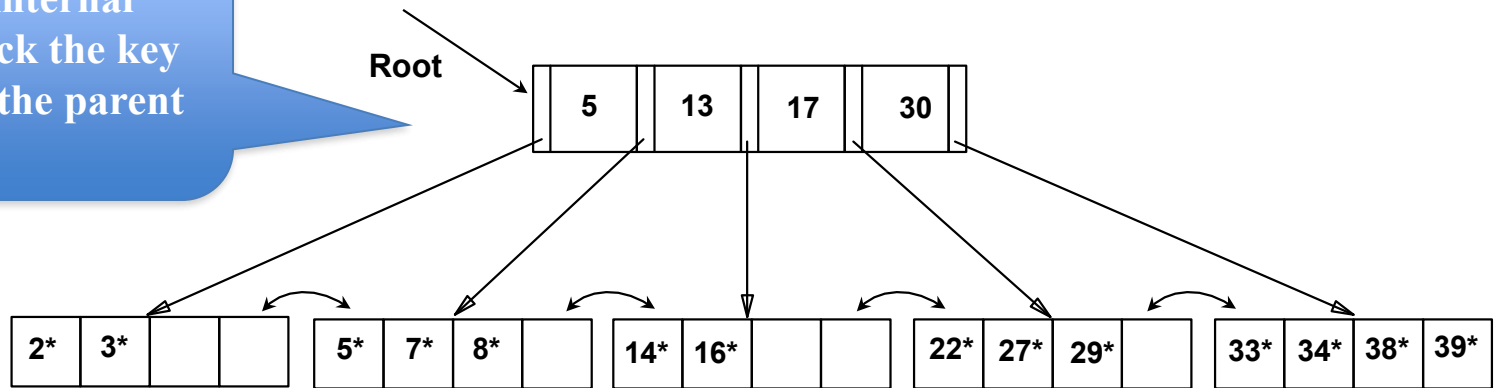


Deleting 24* ...

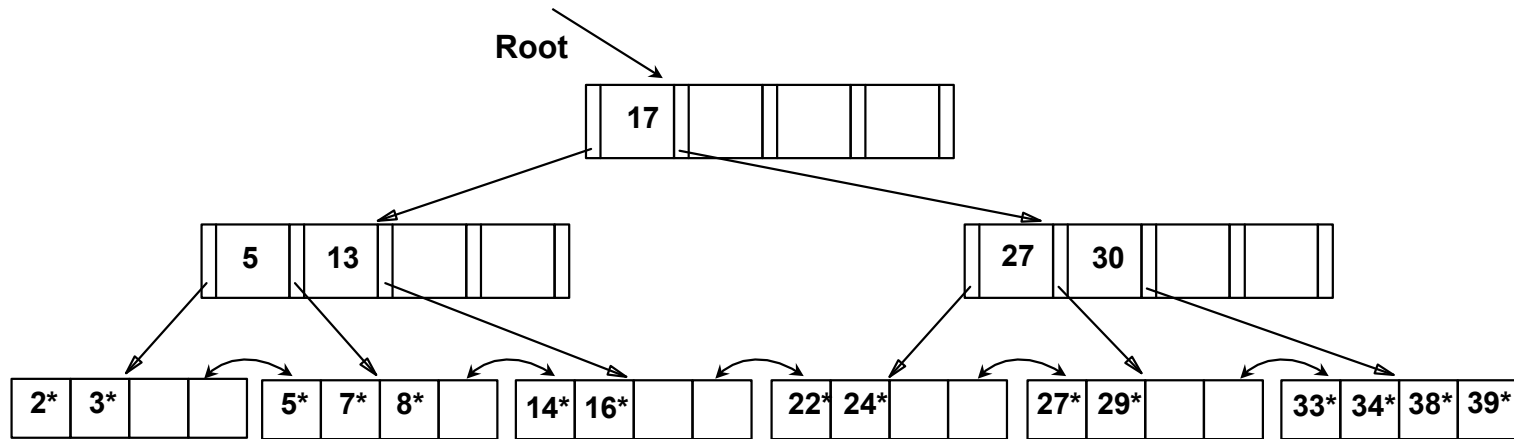


Notice:

When merging internal nodes...you get back the key that you passed to the parent



Deleting 24* ...

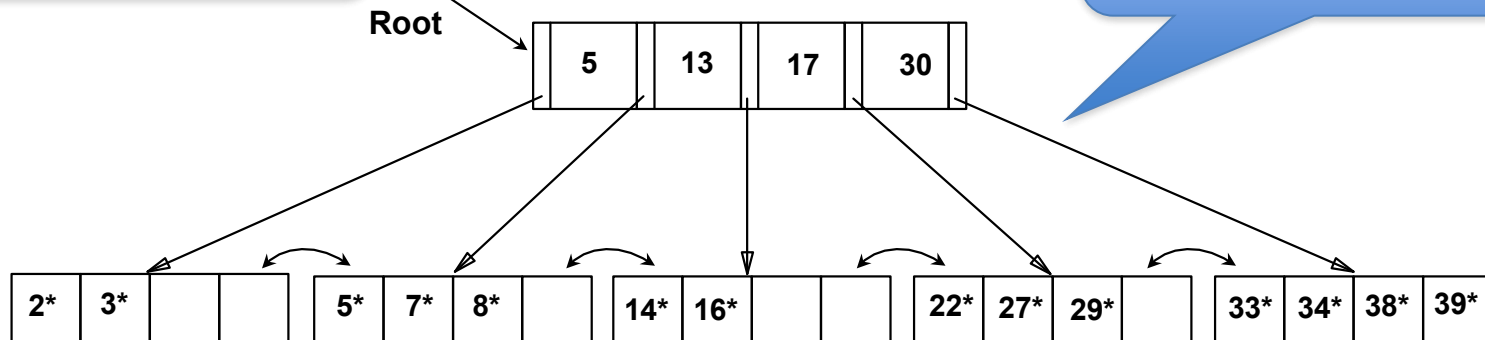


Notice:

Deletion still keeps the tree balanced

Notice:

The tree height may be reduced

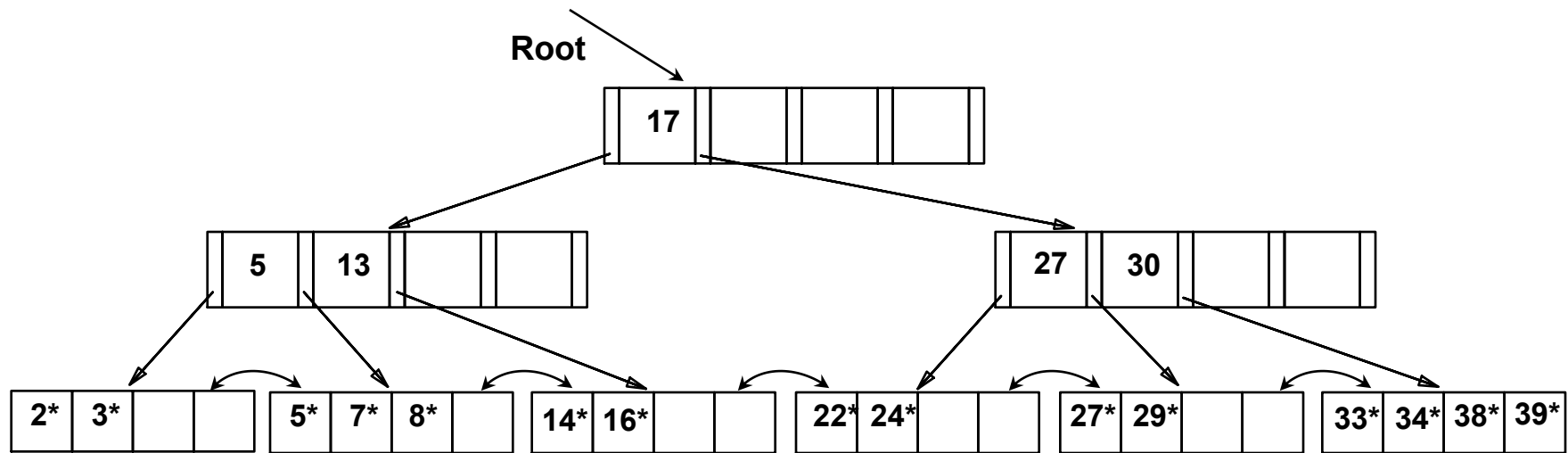


Back to Deletion Algorithm

- Start at root, find **leaf** L where entry belongs. (*Searching*)
- Remove the entry.
 - If L is at least half-full, *done!* 😊
 - If L has less than **the minimum** entries,
 - Try to re-distribute, borrowing from sibling (*adjacent node with same parent as L*).
 - If re-distribution fails, merge L and sibling.
- If merge occurred, must delete entry (pointing to L or sibling) from parent of L .
- Merge could propagate to root, decreasing height.

More Practical Deletion:

Deleting 24* ...



- When deleting 24*, allow its node to be **under** utilized
- After some time with more insertions
 - Most probably more entries will be added to this node
- **If many nodes become under-utilized → Re-build the index**

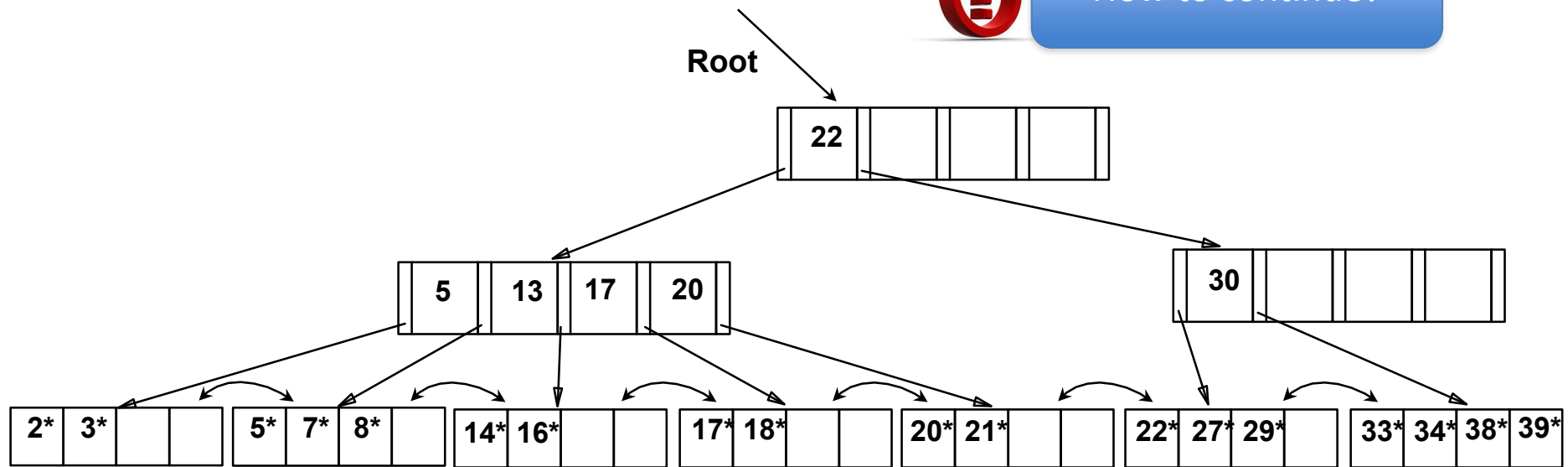
Example of **Non-leaf** Re-distribution

- Assume in the middle of deleting a key we are have the tree below.
- Node with key 30 had an entry just deleted and now it is below the minimum threshold**

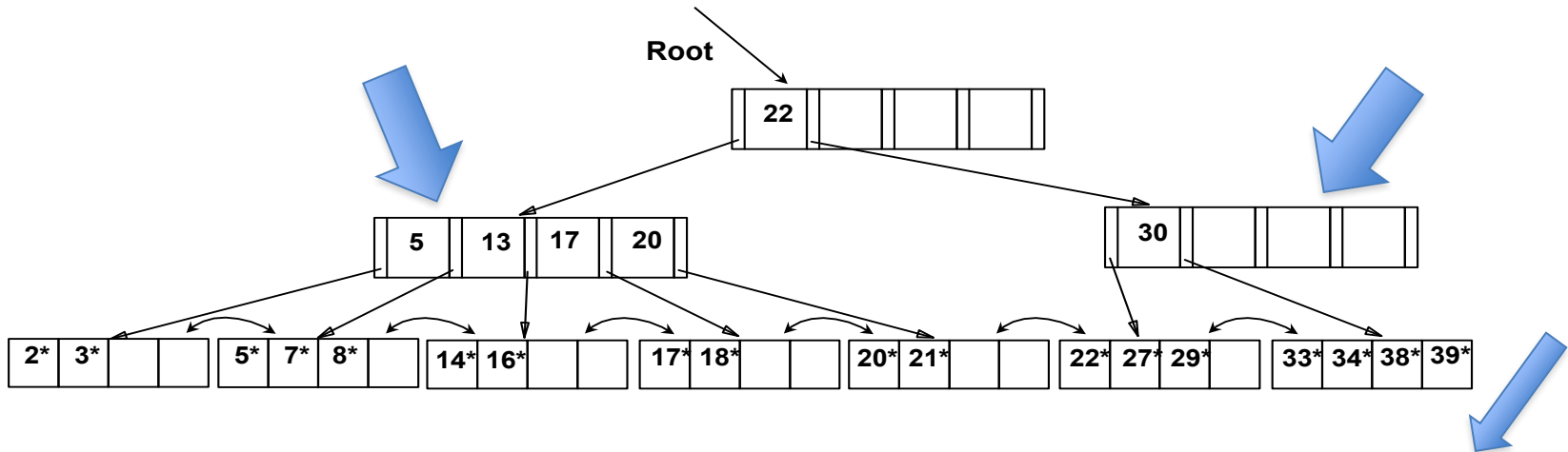


How to continue?

Root



Borrow from your Sibling



- Take the keys of the two nodes + the parent key [5, 13, 17, 20, 22, 30]
- For internal nodes → Do not forget the key that you gave to your parent
- “22” slides to node “30” and “20” takes its place
 - [5, 13, 17] **[20]** [22, 30]
- **Fix the pointers to the leaf nodes**

Summary

- **Easy cases**
 - Key not found
 - Key is found and the leaf node > 50% full
- **Harder case**
 - Get help from either of your siblings (under the same parent)
 - Some key re-distribution
 - Deletion does not propagate upward
- **Hardest case**
 - Could not get help from siblings
 - Need to merge
 - Deletion propagates upward
- Borrowing from sibling (for internal node) ➔ Must involve the key from the parent

Have “fun” with B-trees

- <http://cmudb.io/btree>
- <https://bplustree.app/>